

基于动态热调节与多层传热的低温防护服性能预测及优化

摘要

本文围绕低温防护服的性能预测与优化问题，建立以延长耐寒时间为目标的耦合传热模型，从人体热调节、服装结构及 PCM 放热性能等维度探究耐寒时间的影响因素。

针对问题一，本文建立人体—多层低温防护服耐寒时间预测的六状态动态耦合传热模型。将人体核心、平均皮肤、三层服装及实际皮肤血流作为状态节点，采用 **Pierce 双节点机制**描述低温下的动态血管收缩，通过热阻网络刻画层间导热与外界散热，并利用表观比热法计入 PCM 凝固过程中的潜热释放。采用**自适应 RK45**方法求解耦合常微分方程组，结合下降方向事件函数定位皮肤温度首次达到安全阈值的时刻。在 -40°C 静止无风环境下，平均皮肤温度分别于 **678.621 min** 和 **1000.171 min** 降至 15°C 和 10°C ，据此得到实验者的有效工作时间约为 **678.62 min**。

针对问题二，本文建立有风轻微运动工况下安全返回时间预测的六状态动态耦合传热模型。在问题一模型基础上，采用 **Fanger/ASHRAE** 混合对流机制修正风致外侧散热，将轻微运动引起的衣内换热增强、代谢产热增加及呼吸显热与潜热损失纳入耦合模型。在 -40°C 、风速 3 m/s 和轻微运动条件下，平均皮肤温度降至 15°C 的时间为 **292.756 min**，故实验者应在约 **293 min** 时返回室内，较问题一无风静止工况缩短 **56.860%**。

针对问题三，本文建立防护服外层涂层厚度与耐寒时间优化的变维动态传热规划模型。我们以单层厚度为 0.3 mm 的外层总层数为整数决策变量，通过逐层设置温度节点同步表征增厚引起的热阻与热容变化，并在承重、成本约束下将新增质量折算为站立时间惩罚。采用**有限枚举—自适应 RK45 联合求解方法**：先筛选预算可行层数，再逐一积分变维耦合方程，比较综合目标函数。最优方案为在原外层上新增 3 层，总厚度为 1.2 mm ；此时防护服总质量为 1.500355 kg ，总成本为 544.366950 元，重量修正后的耐寒时间为 **734.815 min**，较问题一净增加 **56.194 min**。

针对问题四，本文建立中间 PCM 放热能力与耐寒时间匹配的六状态动态传热设计模型。保持材料物性不变，引入统一放热倍率同比缩放潜热峰，以平均皮肤温度首次降至 15°C 的时间不低于问题三最优结果为约束，反演最小放热倍率。采用**粗搜索—Brent 标量反演—自适应 RK45 回代验证方法**求解，最小放热倍率为 **28.352528**，即放热能力至少需较原材料提高 **2735.253%**；单位质量潜热由 122.312883 kJ/kg 增至 3467.879499 kJ/kg ，此时耐寒时间达到 **734.815 min**。

本文亮点：将皮肤血流视为独立状态，动态描述人体热调节过程，经检验，该模型具有较好的能量守恒性、数值稳定性和结构优化能力

关键字：人体热调节 相变材料 动态皮肤血流 多层传热 整数优化 成本优化

1 问题重述

1.1 问题背景

低温防护服是保障寒冷环境下人体热安全的重要装备，其作用原理是利用多层材料阻滞人体热量向外界传递，并通过相变材料在降温固化时释放潜热，从而延缓体表温度下降。



图 1

在极地科考和寒冷地区野外作业中，防护效果受材料厚度与热物性、环境风速及人体活动状态等因素影响，而增加厚度又受到负重能力和资金限制。因此，需要预测不同工况下人体可安全进行户外活动的的时间，合理调整材料厚度和相变放热能力，以提高低温防护服的防护效果。

1.2 问题提出

针对低温防护服的传热与性能优化问题，具体研究如下：

问题一：实验者在南极洲长城站附近静止站立，环境温度为 -40°C ，安静、晴天无风。分析人体、防护服与外界之间的传热过程，建立热量传递模型，预测室外坚持时间。

问题二：当长城站外风速为 3m/s 且实验者进行轻微运动时，在问题一模型中引入风速与人体活动的影响，计算实验者必须返回室内的大致时间。

问题三：沿用问题一的静止工况。已知实验者除自身体重外最多可承受 100kg 外部负重；以问题一最长坚持时间为基准，每多承受 0.5kg 使站立时间减少 10s 。新增材料费用不超过原支出的 50% 。在负重与资金约束下优化外层涂层厚度，使室外站立时间最长。

问题四：沿用问题三的厚度与资金条件，通过提高中间功能层的放热能力延长户外活动时间。确定放热能力相对原材料至少提高的比例，使坚持时间不低于问题三结果。

2 模型假设

1. 人体核心、皮肤及三层服装分别视为温度均匀的集中参数节点，热量只沿服装厚度方向传递。
2. 实验者进入低温环境前已经穿着防护服，人体与三层服装达到近似热平衡。
3. 针对 -40°C 极寒低湿环境，假设环境水汽分压 $p_a = 0 \text{ Torr}$ 。
4. 人体质量和等效比热保持不变；静止工况只计基础代谢产热与呼吸散热，忽略颤抖产热、出汗及汗液蒸发。

3 符号说明

表 1 主要符号说明

符号	含义	单位
T_c, T_s	人体核心温度、平均皮肤温度	$^{\circ}\text{C}$
T_1, T_2, T_3	防护服内层、PCM 层和外层节点温度	$^{\circ}\text{C}$
$T_{3,j}$	第 j 个外层涂层节点温度	$^{\circ}\text{C}$
T_{∞}	环境温度	$^{\circ}\text{C}$
α_s	皮肤节点占人体总质量的等效比例	—
$\dot{V}_{bl}, \dot{V}_{bl,eq}$	实际皮肤血流量、目标皮肤血流量	$\text{L}/(\text{m}^2\text{h})$
M_0, M	静止工况、当前活动工况的单位面积代谢产热率	W/m^2
h_{in}, h_{out}	服装内侧、外侧的对流换热系数	$\text{W}/(\text{m}^2\text{K})$
q_{ij}	节点 i 到节点 j 的热流密度	W/m^2
$c_{app}(T)$	PCM 温度相关表观比热	$\text{J}/(\text{kg K})$
v	外界风速	m/s
t_{15}, t_{10}	两个皮肤温度阈值的首次到达时刻	s
N	外层涂层总层数	—

续下页

表 1 主要符号说明（续）

符号	含义	单位
$\delta_{\text{out}}(N)$	层数为 N 时的外层总厚度	m
$J(N)$	综合站立时间	s

4 问题一模型的建立与求解

4.1 问题一分析

问题一旨在预测实验者在 -40°C 静止无风环境中的平均皮肤温度演化，并确定其首次降至 15°C 和 10°C 的时刻。首先，我们将人体核心、皮肤与三层防护服抽象为集中参数节点，沿核心产热、血流输热、服装导热至环境散热的路径建立能量平衡；其中，引入动态皮肤血流描述人体低温下的热调节过程，并采用 PCM 表观比热分析相变过程中的潜热释放，之后，将核心温度、皮肤温度、三层服装温度和实际皮肤血流组成六状态耦合方程，结合初始温度与自然对流边界进行数值积分。最后，输出平均皮肤温度首次下降 15°C 和 10°C 的事件时刻，为有效工作时间和危险阈值时间。

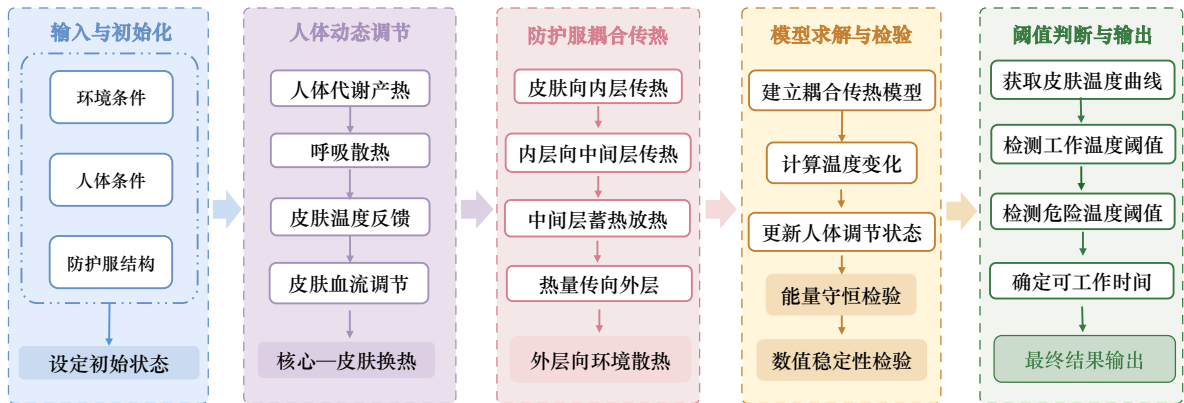


图 2 问题一建模与求解流程图

4.2 总体传热结构

人体核心产生的热量首先传递至皮肤，随后由皮肤进入防护服，并依次经过内层织物、PCM 功能层和外层隔热层，最终散失到低温环境中。因此，主要传热路径可以表示

为

$$T_c \longrightarrow T_s \longrightarrow T_1 \longrightarrow T_2 \longrightarrow T_3 \longrightarrow T_\infty.$$

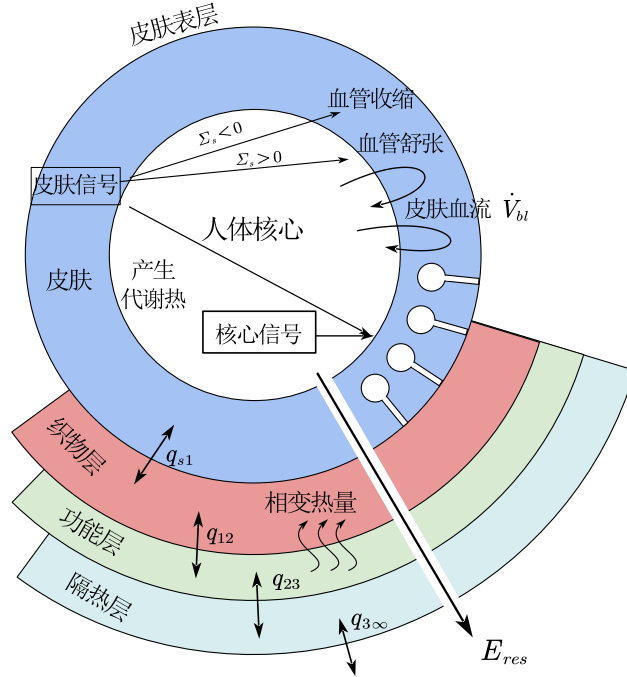


图 3

其中, T_c 和 T_s 分别表示人体核心温度和平均皮肤温度, T_1, T_2, T_3 分别表示内层织物、PCM 功能层和外层隔热层温度, T_∞ 为环境温度。

4.3 基础参数

4.3.1 人体面积与服装材料参数

采用 Stevenson 公式计算人体有效换热面积。对身高 $h = 170 \text{ cm}$ 、体重为 $m_b = 60 \text{ kg}$ 的实验者, 有

$$A_h = 0.0061h + 0.0128w - 0.1529 = 1.6521 \text{ m}^2.$$

衣料面积一般不超过人体表面积的 1.25 倍, 故取

$$A_m = 1.25A_h = 2.0651 \text{ m}^2$$

其中, A_h 用于各界面热流计算, A_m 用于服装质量、热容量、潜热及成本计算。由

$$m_i = \rho_i A_m \delta_i, \quad i = 1, 2, 3.$$

得到三层材料质量分别为

$$m_1 = 0.3007 \text{ kg}, \quad m_2 = 0.4562 \text{ kg}, \quad m_3 = 0.1859 \text{ kg}.$$

4.3.2 代谢及呼吸参数

人体核心是系统内部的主要热源。实验者静止站立时，核心通过代谢产生热量，同时经呼吸排出显热和潜热，扣除呼吸散热后的剩余热量再向皮肤传递。

将人体单位有效体表面积代谢产热率记为 M_0 。为把静止站立工况参数化，本文按 1.2 met 取值，并以 $1 \text{ met} = 58.2 \text{ W/m}^2$ 换算，得到 $M_0 \simeq 70 \text{ W/m}^2$ 。该值是模型的典型活动强度输入，而非实验者的个体实测值。

人体热平衡中还需扣除随代谢率和环境状态变化的呼吸显热与潜热。设 p_a 为环境水汽分压，本文按下列经验关系计算呼吸散热：

$$\begin{aligned} q_{\text{res,lat}} &= 0.0023 M_0 (44 - p_a), \\ q_{\text{res,sens}} &= 0.0014 M_0 (34 - T_\infty), \\ q_{\text{res}} &= q_{\text{res,lat}} + q_{\text{res,sens}}. \end{aligned}$$

第一项表示呼吸潜热损失，第二项表示呼吸显热损失。代入 $p_a = 0 \text{ Torr}$, $M_0 = 70 \text{ W/m}^2$ 和 $T_\infty = -40^\circ\text{C}$ ，分别得到

$$q_{\text{res,lat}} = 7.084 \text{ W/m}^2, \quad q_{\text{res,sens}} = 7.252 \text{ W/m}^2, \quad q_{\text{res}} = 14.336 \text{ W/m}^2.$$

核心剩余热量通过核心-皮肤热流 q_{cs} 输送至皮肤。

4.4 人体动态热调节模型

4.4.1 皮肤血流动态调节与核心-皮肤传热模型

近期人体-防护服一体化研究表明，将人体核心-皮肤热调节与服装传热边界联立，可用于预测防护服穿着者的核心和皮肤温度响应 [2]。因此本文采用核心-皮肤双节点思想作为人体热调节基础，并进一步把实际皮肤血流设为动态状态。

皮肤温度下降会引起外周血管收缩，实际皮肤血流随之减小，并同时改变核心-皮肤传热能力和人体热学分区。该调节过程可概括为

$$T_s \longrightarrow \dot{V}_{bl,\text{eq}} \longrightarrow \dot{V}_{bl} \longrightarrow G_{cs}, \alpha_s \longrightarrow q_{cs}, C_c, C_s.$$

(1) 目标皮肤血流。Pierce 双节点模型同时考虑核心暖信号和皮肤冷信号对血流的调节，其中皮肤冷信号为 $\max(0, 33.7 - T_s)$ ， 33.7°C 为中性皮肤温度。针对持续低温暴

露，本文仅保留皮肤冷信号对应的血管收缩项，并取血管收缩系数 0.5。将当前皮肤温度对应的目标皮肤血流记为 $\dot{V}_{bl,eq}$ ，并为了避免持续冷刺激下血流降为非物理解，设置生理下限 $0.5 \text{ L}/(\text{m}^2\text{h})$ ，得到

$$\dot{V}_{bl,eq}(T_s) = \max \left\{ 0.5, \frac{6.3}{1 + 0.5 \max(0, 33.7 - T_s)} \right\}. \quad (1)$$

(2) 实际血流动态。目标血流表示人体趋向达到的调节状态，实际血管收缩存在有限响应时间，因此采用一阶动态过程描述这一响应。本文以 $\tau_{bl} = 173 \text{ s}$ 作为基准响应时间，并在后文进行结构敏感性检验。据此将实际皮肤血流 $\dot{V}_{bl}$ 作为第六个动态状态，建立

$$\frac{d\dot{V}_{bl}}{dt} = \frac{\dot{V}_{bl,eq}(T_s) - \dot{V}_{bl}}{\tau_{bl}}. \quad (2)$$

(3) 核心-皮肤传热。核心向皮肤传热包括组织导热和血液循环输热，其等效传热系数与相应热流密度为

$$G_{cs} = 5.28 + 1.163\dot{V}_{bl}, \quad q_{cs} = G_{cs}(T_c - T_s). \quad (3)$$

其中，5.28 表示本文参数化的组织基础导热项， $1.163\dot{V}_{bl}$ 描述随皮肤血流变化的循环输热项。

(4) 核心-皮肤热容量分区。实际皮肤血流不仅影响核心向皮肤的传热，还会改变核心与皮肤所承担的等效热容量。本文将人体组织热容 $0.97 \text{ Wh}/(\text{kg K})$ 换算为等效比热 $c_b = 3490 \text{ J}/(\text{kg K})$ 。记皮肤节点所占的等效热学质量比例为 α_s ，并按血流量采用如下动态分区关系：

$$\alpha_s = 0.0417737 + \frac{0.7451832}{\dot{V}_{bl} + 0.585417}. \quad (4)$$

相应地，核心与皮肤节点的等效热容量分别为

$$C_c = (1 - \alpha_s)m_b c_b, \quad C_s = \alpha_s m_b c_b.$$

由于 α_s 随 $\dot{V}_{bl}$ 减小而增大，寒冷环境下皮肤血流降低时皮肤及外周组织节点的等效热容量增大，核心节点的等效热容量相应减小。

4.4.2 动态热学分区的能量守恒修正

由于实际皮肤血流 $\dot{V}_{bl}$ 是动态状态， α_s 及 C_c, C_s 也随时间变化，进一步考虑热学区变化对能量守恒的影响，将热学区变化等效为核心与皮肤节点之间的质量重新分配。

由 $\alpha_s(\dot{V}_{bl})$ 及血流动态方程, 根据链式法则得到

$$\dot{\alpha}_s = -\frac{0.7451832}{(\dot{V}_{bl} + 0.585417)^2} \frac{\dot{V}_{bl,eq}(T_s) - \dot{V}_{bl}}{\tau_{bl}},$$

并定义由核心指向皮肤的等效质量变化率为

$$\dot{m}_{cs} = m_b \dot{\alpha}_s.$$

其中, $\dot{m}_{cs} > 0$ 表示等效热学质量由核心节点重新划入皮肤节点, $\dot{m}_{cs} < 0$ 表示反向重新分配。

重新分配的等效质量同时携带显热。取统一参考温度 $T_{\text{ref}} = 0^\circ\text{C}$, 并采用上游节点温度计算其焓流, 可得

$$\dot{H}_m = \begin{cases} c_b(T_c - T_{\text{ref}})\dot{m}_{cs}, & \dot{m}_{cs} \geq 0, \\ c_b(T_s - T_{\text{ref}})\dot{m}_{cs}, & \dot{m}_{cs} < 0. \end{cases}$$

该焓流将在核心和皮肤两个节点的能量方程中以大小相等、方向相反的形式计入, 从而补偿动态热学分区引起的能量转移, 保证人体内部总能量守恒。

4.4.3 核心-皮肤能量平衡模型

将皮肤向服装内层输出的热流密度记为 q_{s1} , 故核心和皮肤节点的净输入功率为

$$F_c = A_h(M_0 - q_{\text{res}} - q_{cs}), \quad F_s = A_h(q_{cs} - q_{s1}).$$

核心与皮肤的显热储能分别为

$$U_c = C_c(T_c - T_{\text{ref}}), \quad U_s = C_s(T_s - T_{\text{ref}}).$$

将质量交换焓流按相反方向计入两个控制体, 能量守恒写为

$$\frac{dU_c}{dt} = F_c - \dot{H}_m, \quad \frac{dU_s}{dt} = F_s + \dot{H}_m.$$

对上述变热容量储能求导, 可得温度方程

$$(\dot{T}_c, \dot{T}_s) = \begin{cases} \left(\frac{F_c}{C_c}, \frac{F_s + m_b c_b (T_c - T_s) \dot{\alpha}_s}{C_s} \right), & \dot{\alpha}_s \geq 0, \\ \left(\frac{F_c + m_b c_b (T_c - T_s) \dot{\alpha}_s}{C_c}, \frac{F_s}{C_s} \right), & \dot{\alpha}_s < 0. \end{cases} \quad (5)$$

4.5 三层防护服传热模型

4.5.1 层间热阻网络

皮肤传出的热量依次经过内层织物、PCM 功能层和外层隔热层，最终由服装外表面散向环境。静止站立时的内侧换热系数为 $h_{\text{in}} = 3.0 \text{ W}/(\text{m}^2\text{K})$ ；无风条件下，外侧自然对流系数为

$$h_{\text{out}} = 2.38|T_3 - T_{\infty}|^{0.25}.$$

将各层材料的平均温度取在层中心。根据一维导热热阻 $r = \delta/k$ 与对流热阻 $r = 1/h$ ，相邻温度节点之间仅经过各材料的半层厚度，且各段热阻串联，因此有

$$\begin{aligned} r_{s1} &= \frac{1}{h_{\text{in}}} + \frac{\delta_1}{2k_1}, & r_{12} &= \frac{\delta_1}{2k_1} + \frac{\delta_2}{2k_2}, \\ r_{23} &= \frac{\delta_2}{2k_2} + \frac{\delta_3}{2k_3}, & r_{3\infty} &= \frac{\delta_3}{2k_3} + \frac{1}{h_{\text{out}}}. \end{aligned} \quad (6)$$

相应热流密度依次为

$$\begin{aligned} q_{s1} &= \frac{T_s - T_1}{r_{s1}}, & q_{12} &= \frac{T_1 - T_2}{r_{12}}, \\ q_{23} &= \frac{T_2 - T_3}{r_{23}}, & q_{3\infty} &= \frac{T_3 - T_{\infty}}{r_{3\infty}}. \end{aligned} \quad (7)$$

4.5.2 PCM 表观热容模型

PCM 在相变温区内凝固放热，需要将潜热并入温度相关的表观比热。处理过程概括为

$$\text{原始 DSC} \longrightarrow \text{扣除显热基线} \longrightarrow q_L(T) \longrightarrow \phi(T), \xi(T) \longrightarrow c_{\text{app}}(T).$$

个体冷热调节服装可通过相变单元实现被动式热量缓冲 [3]，防护服与相变蓄冷材料耦合的实验研究也表明，PCM 的热容与相变焓会直接影响整装热响应 [4]。因此，本文不把 PCM 仅作为常比热织物，而是根据附件 DSC 曲线建立温度相关的储能项。

将原始质量归一化热流记为 $q(T)$ ，以数据温区首尾点连线作为显热基线 $b(T)$ ，则纯相变峰为

$$q_L(T) = \max\{|q(T)| - b(T), 0\}.$$

近期个体防护用相变材料研究使用 DSC 测试相变温度与潜热，并据此评价其服装应用性能 [5]；DSC 参数研究进一步表明，测试与处理条件会显著影响所得热物性 [6]。为避免普通三次样条在离散放热峰附近产生过冲，本文对 $q_L(T)$ 采用 PCHIP 保形插值。由无相变区 $|q_{\text{base}}| = c_2\beta/60000$ 反演扫描速率，得到

$$\beta = \frac{60000|q_{\text{base}}|}{c_2} \simeq 10 \text{ K/min}.$$

设 DSC 数据温区为 $[T_a, T_b]$ 。为描述潜热在温度区间内的释放位置，将纯相变峰归一化为潜热分布函数

$$\phi(T) = \frac{q_L(T)}{\int_{T_a}^{T_b} q_L(\theta) d\theta}, \quad \int_{T_a}^{T_b} \phi(T) dT = 1.$$

降温凝固时，将温度由 T_b 降至当前温度 T 所完成的相变比例记为

$$\xi(T) = \int_T^{T_b} \phi(\theta) d\theta, \quad \frac{d\xi}{dT} = -\phi(T).$$

其中， $T \geq T_b$ 时取 $\xi = 0$ ， $T \leq T_a$ 时取 $\xi = 1$ 。对相变峰积分得到单位质量潜热

$$L = \frac{60}{\beta} \int_{T_a}^{T_b} q_L(T) dT \approx 122.3 \text{ kJ/kg}.$$

将 PCM 层累计释放的潜热记为 $Q_{\text{lat}} = m_2 L \xi(T_2)$ 。在相变温区内，其释放功率为

$$\dot{Q}_{\text{lat}} = m_2 L \frac{d\xi}{dt} = -m_2 L \phi(T_2) \dot{T}_2.$$

PCM 节点的显热平衡为

$$m_2 c_2 \dot{T}_2 = A_h (q_{12} - q_{23}) + \dot{Q}_{\text{lat}}.$$

将潜热释放功率移至储能项，得到

$$m_2 [c_2 + L \phi(T_2)] \dot{T}_2 = A_h (q_{12} - q_{23}).$$

因此定义表观比热；将 L 换算为 J/kg 并代入其 DSC 积分式，可写成

$$c_{\text{app}}(T) = c_2 + L \phi(T) = c_2 + \frac{60000}{\beta} q_L(T). \quad (8)$$

数据温区外 $\phi(T) = 0$ ，故取 $c_{\text{app}}(T) = c_2 = 2400 \text{ J/(kg K)}$ 。等效热容法可把固液相变潜热统一写入温度方程，其热容曲线的连续性和相变区间识别会影响数值稳定性与预测精度 [7]。复合相变调温纺织品的现有研究也强调，应在材料热物性与整装热调节效果之间建立可计算联系 [8]。

图中左图展示了原始 DSC 信号经显热基线分离后得到的潜热峰，右图展示了该潜热峰换算得到的温度相关表观比热，可以看出 PCM 的潜热主要集中在相变温区内释放。

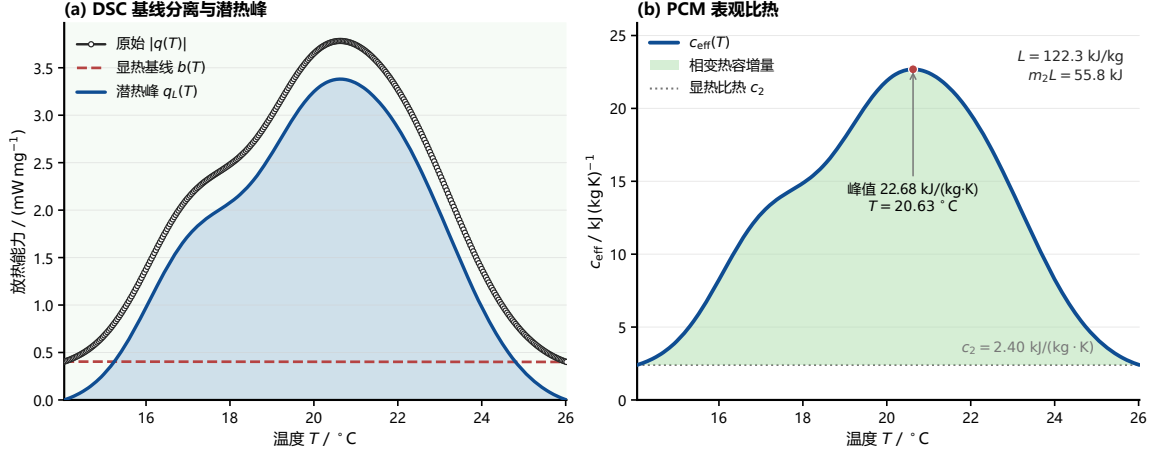


图 4 DSC 基线分离与 PCM 表观比热曲线

4.5.3 三层材料能量平衡

各服装节点满足流入热量减去流出热量等于储能变化，因此三层材料的能量方程为

$$\begin{aligned} m_1 c_1 \dot{T}_1 &= A_h (q_{s1} - q_{12}), \\ m_2 c_{app}(T_2) \dot{T}_2 &= A_h (q_{12} - q_{23}), \\ m_3 c_3 \dot{T}_3 &= A_h (q_{23} - q_{3\infty}). \end{aligned} \quad (9)$$

4.6 六状态耦合模型

取状态向量

$$\mathbf{x}(t) = [T_c, T_s, T_1, T_2, T_3, \dot{V}_{bl}]^T,$$

联立式(1)–(2)、式(5)以及式(6)–(9)，得到六状态耦合模型

$$\left\{ \begin{aligned} (\dot{T}_c, \dot{T}_s) &= \begin{cases} \left(\frac{F_c}{C_c}, \frac{F_s + m_b c_b (T_c - T_s) \dot{\alpha}_s}{C_s} \right), & \dot{\alpha}_s \geq 0, \\ \left(\frac{F_c + m_b c_b (T_c - T_s) \dot{\alpha}_s}{C_c}, \frac{F_s}{C_s} \right), & \dot{\alpha}_s < 0, \end{cases} \\ \dot{T}_1 &= \frac{A_h (q_{s1} - q_{12})}{m_1 c_1}, \\ \dot{T}_2 &= \frac{A_h (q_{12} - q_{23})}{m_2 c_{app}(T_2)}, \\ \dot{T}_3 &= \frac{A_h (q_{23} - q_{3\infty})}{m_3 c_3}, \\ \frac{d\dot{V}_{bl}}{dt} &= \frac{\dot{V}_{bl,eq}(T_s) - \dot{V}_{bl}}{\tau_{bl}}. \end{aligned} \right. \quad (10)$$

4.7 初始与边界条件

实验者在 $t = 0$ 时进入低温环境，外界温度变为 $T_\infty = -40^\circ\text{C}$ 。测试初始人体温度为 37°C ，故取

$$T_c(0) = T_s(0) = 37^\circ\text{C}.$$

服装三层节点初温与人体相同，取

$$T_1(0) = T_2(0) = T_3(0) = 37^\circ\text{C}.$$

此时皮肤冷信号为零，本文将目标皮肤血流初始化为中性值 $6.3\text{ L}/(\text{m}^2\text{h})$ 。因此初始状态向量为

$$\mathbf{x}(0) = [37, 37, 37, 37, 37, 6.3]^\top.$$

4.8 模型求解

Step 1: 输入模型参数与初始状态。将人体和服装参数、四段热阻关系、PCM 表观比热函数以及初始状态 $\mathbf{x}(0)$ 传入六状态模型，设置环境温度在 $t = 0$ 时降至 -40°C 。

Step 2: 计算六状态方程右端。每次计算先由当前 T_s 得到目标血流和实际血流变化率，再由实际血流更新 G_{cs} 、 α_s 与 $\dot{\alpha}_s$ 。随后依次计算核心-皮肤热流、四段服装热流及当前 T_2 对应的 $c_{\text{app}}(T_2)$ ，最终得到人体两节点、服装三节点和实际血流共六个状态的导数。基准计算采用自适应 RK45 方法，设置 $\text{rtol} = 10^{-7}$ 、 $\text{atol} = 10^{-9}$ 、最大步长 5 s ，计算上限为 24 h 。自适应 Runge-Kutta 初值问题求解框架能够将误差控制、连续延拓与事件根定位结合起来 [9]，适合本文同时含快速 PCM 相变和慢速人体降温的多时间尺度过程。

Step 3: 定位阈值事件并复核精度。定义下降方向事件

$$g_{15}(t, \mathbf{x}) = T_s - 15, \quad g_{10}(t, \mathbf{x}) = T_s - 10. \quad (11)$$

由求解器连续延拓得到两个事件的首次过零时刻；检测到 10°C 事件后终止积分。加严计算仍采用 RK45 方法， $\text{rtol} = 10^{-9}$ 、 $\text{atol} = 10^{-11}$ ，最大步长减小为 1 s ，用于复核事件定位精度。

4.9 结果与分析

4.9.1 温度响应与阈值时间

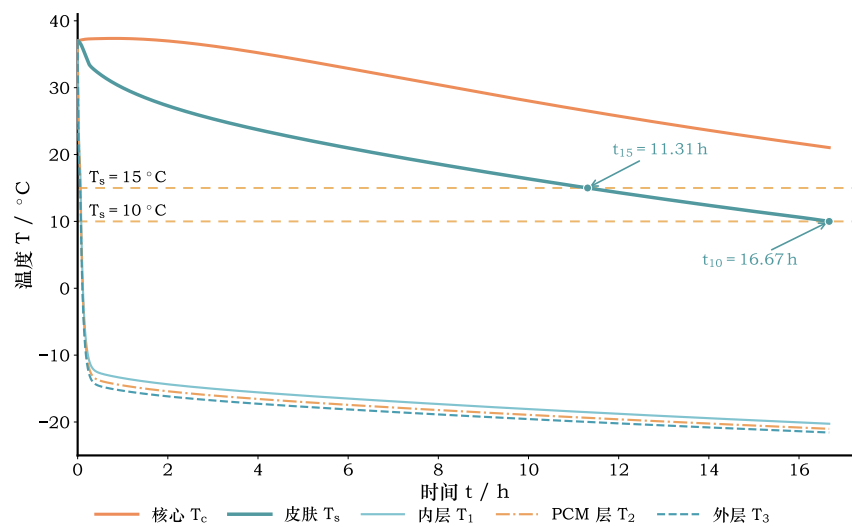


图 5 极寒环境下人体与防护服各节点温度响应

表 2 人体低温阈值

皮肤阈值/°C	时刻/s	时刻/min	核心温度/°C	实际血流 L/(m ² h)	α_s
15	40717.233	678.621	26.511	0.6101	0.6651
10	60010.271	1000.171	21.036	0.5000	0.7283

平均皮肤温度约在 678.62 min，即 11.31 h 后降至 15 °C，此后正常工作开始受到影响；约在 1000.17 min，即 16.67 h 后降至 10 °C，达到危险阈值。

皮肤温度在进入低温环境后立即下降，核心温度则先在约 48.3 min 时小幅升至 37.370 °C，随后缓慢降低。外界冷负荷首先作用于皮肤，人体核心依靠自身热容量和代谢产热延缓降温，因此皮肤温度与核心温度存在时间尺度的差异。冷扰动由外层向内层传播，因此三层服装节点在暴露初期迅速降温，并在全时程保持 $T_1 > T_2 > T_3$ ，对应。

4.9.2 血流收缩与热容量重分配

随着皮肤降温，目标皮肤血流迅速下降，而实际血流受 $\tau_{bl} = 173\text{ s}$ 的动态响应限制，在暴露初期表现出明显滞后，随后逐渐趋近目标值。当皮肤温度降至 15 °C 和 10 °C 时，实际血流分别降至 0.6101 L/(m²h) 和 0.5000 L/(m²h)。

与此同时，皮肤等效质量比例 α_s 随血流收缩由初始 0.15 增至 0.7283。因此，血流收缩一方面降低核心-皮肤传热能力，减少核心向外周的热量输送；另一方面改变核心与皮肤的等效热容量分配，从而形成对低温暴露的动态调节。

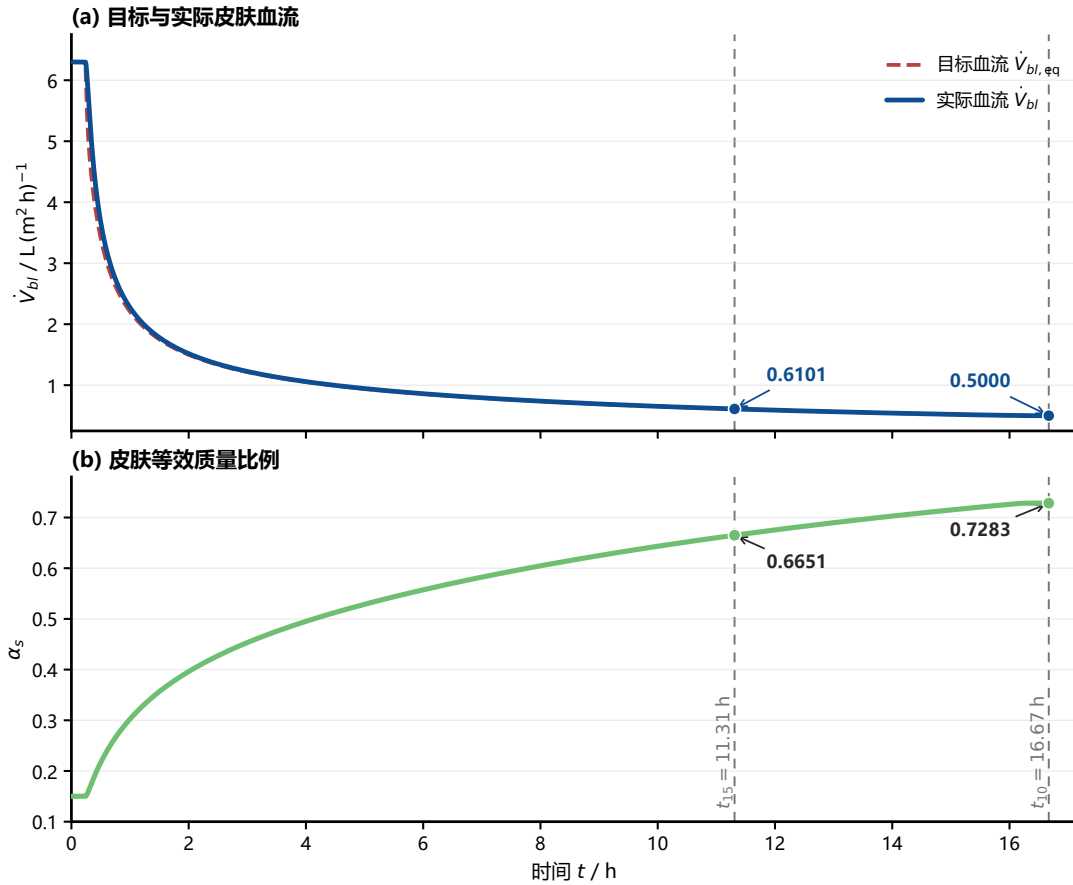


图 6 皮肤血流动态调节与等效质量比例演化

4.9.3 PCM 相变热容与潜热释放过程

PCM 在低温暴露初期存在潜热释放过程，其表观比热与累计潜热随时间发生变化。PCM 约在 60 s 时进入主要相变区间，并于约 225 s 时基本完成。相变过程中，表观比热先升高后降低，在约 130 s 时达到峰值 22.680 kJ/(kg·K)；与此同时，累计潜热持续增加，最终稳定在约 55.8 kJ。由此可见，PCM 的主要作用集中在进入极寒环境后的最初数分钟，通过释放有限潜热缓冲初始冷冲击；相变结束后，长期耐寒过程主要由人体产热及防护服隔热性能共同决定。

4.9.4 逐级传热路径的时间演化

热量由人体经皮肤及三层服装向环境传递，在 $t = 0$ 时，人体与服装各节点均为 37℃，内部温差为零，而外层与环境存在 77 K 温差，使 $q_{3\infty}$ 瞬时升至 532.184 W/m²。随后冷扰动由外向内传播，各段热流依次建立。除初始时刻外，前 10 min 内总体满足 $q_{3\infty} > q_{23} > q_{12} > q_{s1} > q_{cs}$ ，其中 q_{12} 的阶段性变化反映了 PCM 相变潜热对热流传递的缓冲作用。

约 30 min 后， q_{s1} 、 q_{12} 、 q_{23} 和 $q_{3\infty}$ 均接近 133 W/m²，表明服装传热链逐渐进入准稳

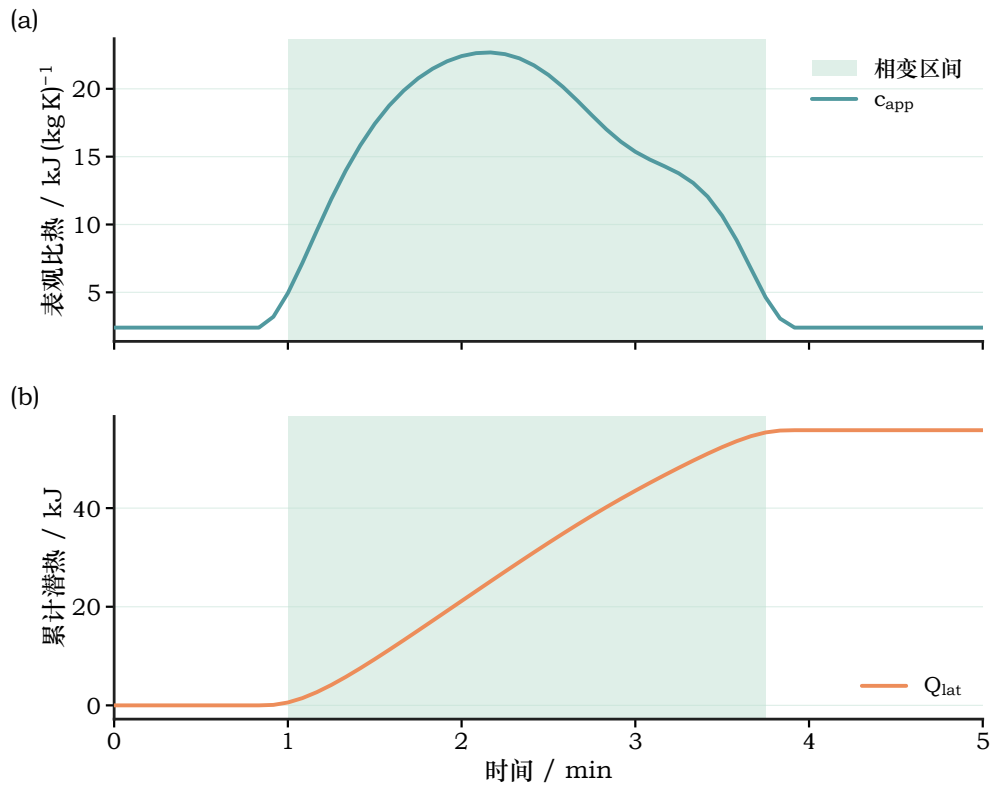


图 7 PCM 表观比热与累计潜热释放的时间过程

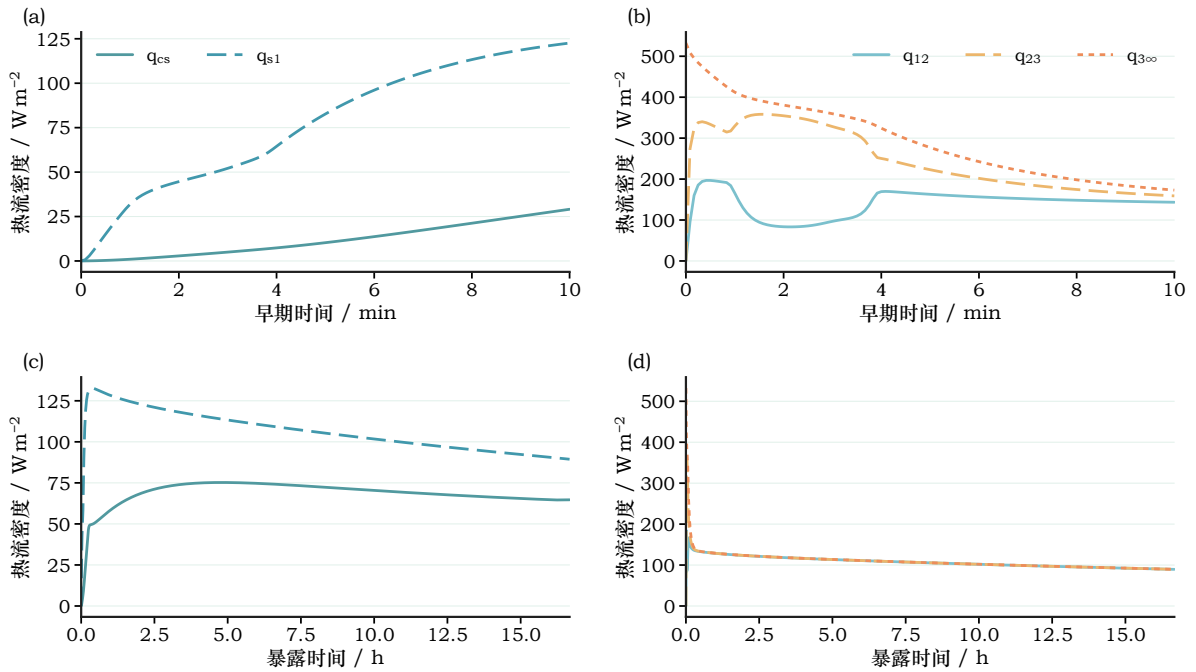


图 8 人体-防护服逐级传热路径的热流密度时间演化

态；此时 q_{cs} 约为 51.1 W/m^2 ，仍受核心-皮肤温差及动态血流调节影响。总体来看，外界冷冲击首先消耗服装与 PCM 储热；服装侧进入准稳态后，环境散热逐渐转由人体显热储备和代谢产热经皮肤与服装持续补给。

4.10 模型检验与敏感性分析

4.10.1 血流迟滞的结构检验

改变 τ_{bl} 所得阈值时间见表 3。当 $\tau_{bl} = 1 \text{ s}$ 时，六状态结果已接近五状态模型，验证了 $\tau_{bl} \rightarrow 0$ 时恢复准稳态血流的退化关系。

表 3 皮肤血流响应时间常数的结构敏感性

τ_{bl}/s	t_{15}/s	t_{10}/s
1	40725.231	59983.661
60	40722.456	59992.676
173	40717.233	60010.271
200	40716.006	60014.542

与五状态准稳态模型相比，六状态主模型的 t_{15} 提前 7.91 s ， t_{10} 延后 26.99 s ，相对变化分别为 -0.019% 和 0.045% 。动态血流使早期实际血流高于目标值，同时减小皮肤等效热容量的增长速度；两种作用在整个暴露过程中部分抵消，因此加入血流迟滞改善了生理过程描述，但没有改变本问的小时量级结论。

在 $60 \sim 200 \text{ s}$ 范围内， t_{15} 和 t_{10} 相对主模型的变化均不超过 0.03% 。响应变慢时，实际血流在降温初期维持较高水平，核心向皮肤输热增强；与此同时 α_s 增长较慢，皮肤等效热容量较小。两条路径方向相反，使最终阈值对 τ_{bl} 的变化不敏感。

4.10.2 换热与产热参数敏感性

对六状态主模型的基础代谢率、内侧换热系数和外侧自然对流倍率分别作 $\pm 10\%$ 扰动，得到表 4。

表 4 六状态主模型的单因素敏感性

参数	-10% 时 Δt_{15}	+10% 时 Δt_{15}	-10% 时 Δt_{10}	+10% 时 Δt_{10}
h_{in}	+16.06%	-11.57%	+16.90%	-11.93%
M_0	-8.28%	+10.21%	-9.76%	+12.49%
λ_{out}	+7.31%	-5.68%	+7.70%	-5.92%

内侧换热系数的影响最大： h_{in} 增大使皮肤至内层的热阻减小，两个阈值均明显提前。代谢率通过改变净产热直接控制耐寒时间，外侧自然对流倍率则通过改变服装向环境的散热影响阈值。服装内、外侧换热条件的不确定性对结论的影响明显大于血流响应时间常数，应在实际应用中优先标定。

4.10.3 集中参数近似检验

初始工况下内层和外层 Biot 数分别约为 0.031 和 0.040，均小于集中参数法常用判据 $Bi < 0.1$ ；三层材料的非相变热扩散特征时间约为 7.2 s、3.5 s 和 2.8 s。服装层厚度处于亚毫米量级，层内温度调整远快于人体降温过程，因此每层可以采用一个平均温度节点。

5 问题二模型的建立与求解

5.1 问题分析

问题二要求在问题一模型的基础上，进一步考虑空气流动和人体运动对传热过程的影响。我们需要修正人体表面—内层织物和外层隔热层—外部环境两个空气边界层的对流换热条件，同时，轻微运动还会提高人体代谢产热，并相应增加呼吸显热与潜热损失。最终得到该情况下的室外停留时间。在此基础上重新求解人体—防护服耦合传热过程，得到实验者在该工况下的室外安全停留时间。

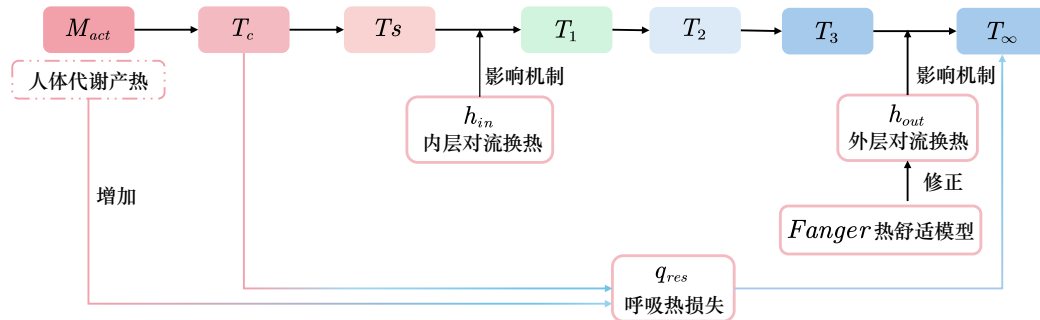


图9 问题二优化机制图

5.2 有风轻微运动条件下低温防护服耦合传热模型的建立

5.2.1 轻微运动下的内侧对流修正

轻微运动会使皮肤与内层织物之间的空气层发生扰动。轻微运动工况下，将内侧等效对流换热系数取为

$$h_{in} = 4.0 \text{ W}/(\text{m}^2\text{K}).$$

从皮肤表面到内层织物中心，热量首先经过衣内空气层对流，再经过内层织物的半层导热。由于两段传热过程串联，单位面积总热阻为

$$r_{s1} = \frac{1}{h_{in}} + \frac{\delta_1}{2k_1}. \quad (12)$$

根据热流密度等于温差除以热阻，可得皮肤向内层织物的热流密度：

$$q_{s1} = \frac{T_s - T_1}{r_{s1}}. \quad (13)$$

5.2.2 外界风作用下的外侧混合对流

由于长城站存在 $v = 3 \text{ m/s}$ 的外部风速，外层隔热层与环境之间的换热需要使用强制对流经验式进行定量表示。织物传热实验表明，风速提高会同时增大瞬态与稳态热流密度 [10]，因此本文把题设风速显式写入外侧换热边界，取：

$$h_{forced} = 12.1\sqrt{v}, \quad (14)$$

该关系表征外部气流不断破坏服装表面的热边界层，使边界层厚度减小，从而增强服装表面与环境之间的对流换热。

自然对流和强制对流可能同时存在。按照 Fanger/ASHRAE 对流换热模型，当二者同时存在时，取其中较大的一个作为实际对流换热系数：

$$h_{out} = \max \{h_{nat}, h_{forced}\}, \quad (15)$$

意味着实际换热由占主导地位的对流机制控制。

从外层隔热材料中心到环境，传热过程由外层材料半层导热和表面对流串联组成，故单位面积总热阻为

$$r_{3\infty} = \frac{\delta_3}{2k_3} + \frac{1}{h_{out}}. \quad (16)$$

最终，外层向环境的热流密度由外层—环境温差除以该总热阻得到：

$$q_{3\infty} = \frac{T_3 - T_{\infty}}{r_{3\infty}}. \quad (17)$$

5.2.3 轻微运动下的代谢与呼吸散热模型

轻微运动会提高人体单位面积代谢率。本文将“站立、轻度活动”参数化为 1.6 met ，并按照 $1 \text{ met} = 58.2 \text{ W/m}^2$ 的关系换算：

$$M_{\text{act}} = 1.6 \times 58.2 \approx 93 \text{ W/m}^2.$$

因此本文令 $M = M_{\text{act}}$ 。该取值用于将题目中的“轻微运动”转化为可计算的代谢强度，是标准活动表中的典型值，并非实验者的个体实测值。

代谢率升高会增加人体内部产热，同时也会提高呼吸散热。为保持与问题一的人体热平衡结构一致，本文同步用活动代谢率重算呼吸热损失。其中呼吸潜热损失由代谢率和空气水汽分压共同决定：

$$q_{\text{res,lat}} = 0.0023M(44 - p_a).$$

呼吸显热损失由代谢率和环境温度共同决定：

$$q_{\text{res,sens}} = 0.0014M(34 - T_{\infty}).$$

上述经验关系使用时 M 以 W/m^2 计， p_a 以 Torr 计， T_{∞} 以 $^{\circ}\text{C}$ 计。由于两项均随 M 变化，修改代谢率后必须同步重新计算呼吸潜热与显热损失。

将两部分相加，得到单位面积呼吸总散热：

$$q_{\text{res}} = q_{\text{res,lat}} + q_{\text{res,sens}}.$$

因此，人体核心节点接收的总代谢功率为

$$\dot{Q}_{\text{met}} = A_h M,$$

扣除呼吸散热后的净热输入为

$$\dot{Q}_{\text{net}} = A_h (M - q_{\text{res}}).$$

5.3 模型求解与结果分析

5.3.1 初始条件、数值方法与事件判定

初始时假设人体与服装已在室内达到相同温度，因此取

$$T_c(0) = T_s(0) = T_1(0) = T_2(0) = T_3(0) = 37^{\circ}\text{C}. \quad (18)$$

初始实际皮肤血流量取静息基准值

$$\dot{V}_{bl}(0) = 6.3 \text{ L}/(\text{m}^2\text{h}). \quad (19)$$

本文采用 `solve_ivp` 对六状态方程组进行数值积分，基准求解器选择 **RK45**，相对误差限和绝对误差限分别设为 10^{-7} 和 10^{-9} ，最大积分步长取 5 s。为识别皮肤温度向下穿越两个阈值的时刻，分别定义事件函数

$$g_{15}(t, \mathbf{x}) = T_s(t) - 15, \quad (20)$$

$$g_{10}(t, \mathbf{x}) = T_s(t) - 10. \quad (21)$$

两个事件函数的检测方向均设为 -1 ，只记录降温过程中的穿越。事件时间由求解器的连续插值确定，而非由每 5 s 输出一次的离散采样点近似读取；若阈值在 24 h 的计算范围内未被触发，则将相应事件记为“未达到”。

5.3.2 返回时间与阈值状态

已有防护服人体试验将核心温度和皮肤温度作为评价液冷型与蓄冷型背心热生理效应的关键指标 [11]。据此，本文同时报告皮肤阈值时间与对应核心温度，而不只给出服装材料节点的温度。

在环境温度为 -40°C 、风速为 3 m/s 且实验者进行轻微运动的条件下，皮肤温度首次下降至 15°C 的时间为

$$t_{15} = 17565.379485 \text{ s} = 292.756325 \text{ min.}$$

因此，在本文模型与安全阈值定义下，实验者应在室外停留约 293 min 时返回室内。

皮肤温度继续下降至 10°C 的时间为

$$t_{10} = 26317.359901 \text{ s} = 438.622665 \text{ min.}$$

除皮肤温度外，阈值时刻的人体核心、PCM 和皮肤血流状态如表 5 所示。

表 5 问题二主工况的阈值时间与关键状态

阈值	时刻/s	时刻/min	$T_c/^\circ\text{C}$	$\dot{V}_{bl}/[\text{L}/(\text{m}^2\text{h})]$	ξ
$T_s = 15^\circ\text{C}$	17565.379	292.756	31.732	0.612	1.000
$T_s = 10^\circ\text{C}$	26317.360	438.623	26.260	0.500	1.000

由表可见，达到 15°C 阈值时核心温度已降至 31.732°C ，实际皮肤血流也已接近血管收缩状态下的低值；达到 10°C 时核心温度进一步降至 26.260°C 。因此， t_{10} 仅用于刻画更低温度风险边界，不能作为建议继续工作的时间。

5.3.3 温度演化、外侧对流与 PCM 放热结果

问题二完整工况下五个温度节点的演化如图 10 所示。

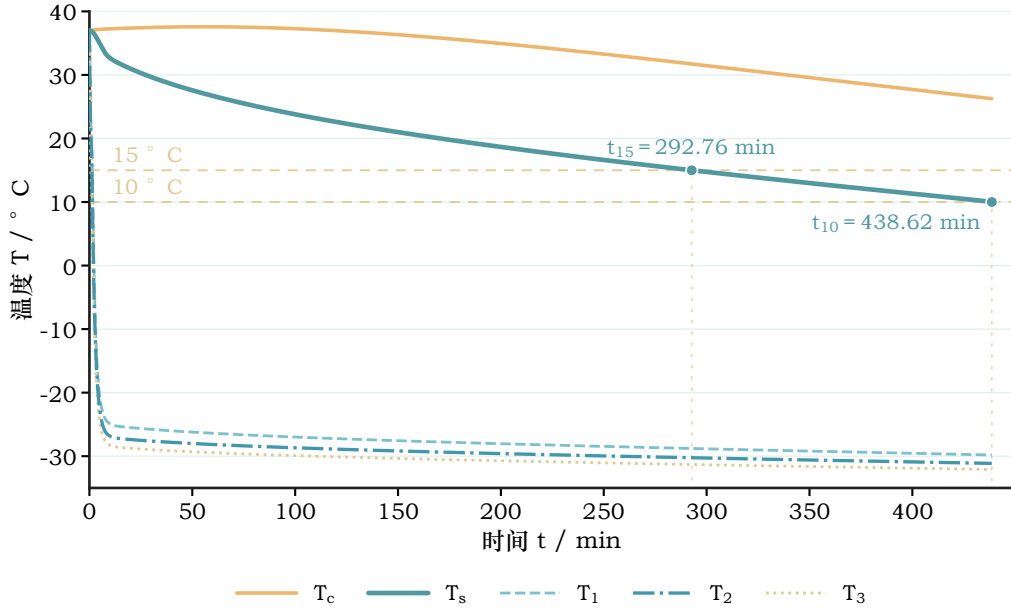


图 10 问题二完整工况下人体与服装温度演化

外层隔热层、PCM 层和内层织物进入极寒环境后迅速降温，并在较短时间内接近各自的准稳态温度；皮肤和核心温度则在人体较大热容量及核心-皮肤换热的共同作用下缓慢下降。

将 $v = 3 \text{ m/s}$ 代入强制对流经验式可得

$$h_{\text{forced}} = 12.1\sqrt{3} = 20.957815 \text{ W}/(\text{m}^2\text{K}). \quad (22)$$

时间序列中自然对流换热系数的最大值为 $7.050171 \text{ W}/(\text{m}^2\text{K})$ ，始终小于强制对流换热系数，因此整个计算时段的外侧换热均由强制对流控制。与问题一相比，服装外表面的对流热阻显著减小，形成了问题二降温加快的主要外部原因。

PCM 节点约在 20 s 时进入相变区，并约在 80 s 时完成相变。由 PCM 质量与单位质量潜热相乘，潜热释放上限为

$$Q_{\text{PCM,max}} = m_2 L = 55802.490579 \text{ J}. \quad (23)$$

两个阈值时刻的相变进度均为 $\xi = 1$ ，说明 PCM 潜热在暴露早期已经全部释放。

5.3.4 外界风、衣内换热与代谢的效应拆分

为识别问题二三项修正的独立作用，设置 Q1、W、I、M 和 Q2 五个工况。其中 Q1 为问题一无风静止基线；W 仅加入外界风；I 仅将内侧对流换热系数提高至 $4.0 \text{ W}/(\text{m}^2\text{K})$ ；M 仅将代谢率提高至 $93 \text{ W}/\text{m}^2$ ；Q2 同时引入全部问题二条件。以 Q1 为基准，第 j 个工况的相对变化率定义为

$$\Delta t_{15}^{(j)} = \frac{t_{15}^{(j)} - t_{15}^{(Q1)}}{t_{15}^{(Q1)}} \times 100\%. \quad (24)$$

各工况结果见表 6 和图 11。

表 6 问题二各修正因素对返回时间的效应拆分

工况	条件变化	t_{15}/min	相对 Q1 变化
Q1	无风, $h_{\text{in}} = 3.0$, $M = 70$	678.621	0
W	仅令 $v = 3 \text{ m/s}$	379.387	-44.094%
I	仅令 $h_{\text{in}} = 4.0 \text{ W}/(\text{m}^2\text{K})$	481.040	-29.115%
M	仅令 $M = 93 \text{ W}/\text{m}^2$	991.325	+46.079%
Q2	同时引入全部问题二条件	292.756	-56.860%

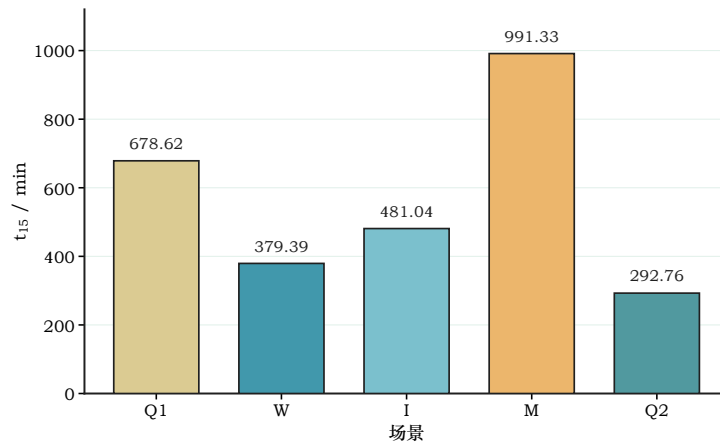


图 11 问题二各修正因素的 t_{15} 效应拆分

单独加入外界风使 t_{15} 缩短 44.094%，单独提高内侧换热系数使其缩短 29.115%，而仅提高代谢率可使其延长 46.079%。完整 Q2 工况相较 Q1 缩短 56.860%，表明运动代谢补热不足以抵消外界风致强制对流和衣内换热增强造成的综合热损失。三种单因素变化率不能直接线性相加，因为热流随节点温差变化，皮肤血流及人体热容量动态调整，PCM 有效热容也随温度变化，Q2 反映的是这些机制非线性耦合后的净效应。

5.3.5 参数敏感性与稳健性分析

为消除参数量纲差异，分别对风速 v 、内侧换热系数 h_{in} 、代谢率 M 和 DSC 扫描速率 β 作 $\pm 10\%$ 单因素扰动。局部敏感性分析可通过重复求解动态 ODE，在基准点附近

比较参数扰动后的输出变化 [12]；本文据此定义中心无量纲敏感度

$$S_{t_{15}}^{(p)} = \frac{t_{15}(1.1p) - t_{15}(0.9p)}{0.2 t_{15}(p)}. \quad (25)$$

计算结果见表 7。

表 7 问题二返回时间的局部无量纲敏感度

参数	$S_{t_{15}}$	$ S_{t_{15}} $	$S_{t_{10}}$	影响方向
h_{in}	-1.395080	1.395080	-1.408709	增大则阈值提前
M	0.489214	0.489214	0.616572	增大则阈值延后
v	-0.135420	0.135420	-0.135291	增大则阈值提前
β	-0.002216	0.002216	-0.001501	增大则阈值略提前

按 $|S_{t_{15}}|$ 排序，局部敏感性依次为内侧换热系数、代谢率、风速和 DSC 扫描速率。效应拆分中风的全局影响大于内侧换热，而局部敏感性中 h_{in} 最高，二者并不矛盾：前者比较从无风到 3 m/s 的工况切换，后者只考察 Q2 基准点附近的 $\pm 10\%$ 小扰动。由于基准工况中强制对流已占主导，风速的小幅改变只通过平方根关系作用于换热系数；相较之下， h_{in} 直接改变皮肤至内层织物的主要串联热阻，因此局部影响更强。

对血流响应时间常数分别取 60 s、173 s 和 200 s，所得返回时间依次为

$$t_{15} = 17572.533 \text{ s}, \quad 17565.379 \text{ s}, \quad 17563.797 \text{ s}. \quad (26)$$

三者的最大差异不足 9 s。这表明返回时间对血流响应时间尺度的合理变化较为稳健，而内侧换热和代谢率的标定更值得优先关注。

5.3.6 能量守恒与求解器收敛验证

为检验各节点方程的整体一致性，将人体核心、皮肤和三层服装的储能相加，定义系统总储能

$$U_{\text{tot}} = U_c + U_s + U_1 + U_2 + U_3, \quad (27)$$

其中 PCM 储能通过表观热容对应的焓积分计算。从初始时刻到 t 的外部理论净输入为代谢产热减去呼吸散热和外层向环境的散热，由此定义能量残差

$$R_E(t) = U_{\text{tot}}(t) - U_{\text{tot}}(0) - \int_0^t A_h (M - q_{\text{res}} - q_{3\infty}) \, d\tau. \quad (28)$$

基准计算的最大绝对能量残差为 1.914783 J，相对能量误差为 3.809929×10^{-7} 。与全过程的能量交换规模相比，该误差可以忽略，说明节点热流、动态质量重分配和 PCM 焓值在数值实现中保持一致。

进一步采用 DOP853 求解器，并将相对误差限、绝对误差限和最大步长分别加严至 10^{-9} 、 10^{-11} 和 1 s。严格解与基准 RK45 的事件时间差为

$$|t_{15}^{\text{DOP853}} - t_{15}^{\text{RK45}}| = 0.001306 \text{ s}, \quad (29)$$

$$|t_{10}^{\text{DOP853}} - t_{10}^{\text{RK45}}| = 0.001332 \text{ s}. \quad (30)$$

该差异远小于论文采用的分钟级报告精度。能量守恒检查验证了方程实现的一致性，双求解器比较验证了事件时间的数值收敛性，两项结果共同支持问题二求解结果的可靠性。

综上，在 -40°C 、3 m/s 风速和轻微运动条件下，实验者的预测返回时间为约 293 min，较问题一无风静止工况缩短 56.86%。该结论适用于本文给定参数和集总传热假设，不应外推为不考虑个体差异、局部风场及其他生理反应时的无条件医学安全时限。

6 问题三模型的建立与求解

6.1 问题三分析

问题三旨在确定承重与成本约束下外层涂层的最优厚度，需要同时衡量增厚产生的保温收益与新增质量造成的站立时间损失。首先，沿用问题一静止、无风工况下的人体-防护服耦合传热模型，将厚度固定为 0.3 mm 的外层总层数作为整数决策变量，并按层设置温度节点，使外层热阻和热容量随层数同步变化。之后计算各方案的总质量与总成本，利用 100 kg 承重上限和原支出 1.5 倍的成本上限筛选可行层数。最后求解各可行方案的传热过程，确定皮肤温度首次降至 15°C 的时刻，同时将新增质量按题目给出的承重关系折算为等效时间代价，并与传热时长综合评价，以确定最优涂层厚度。

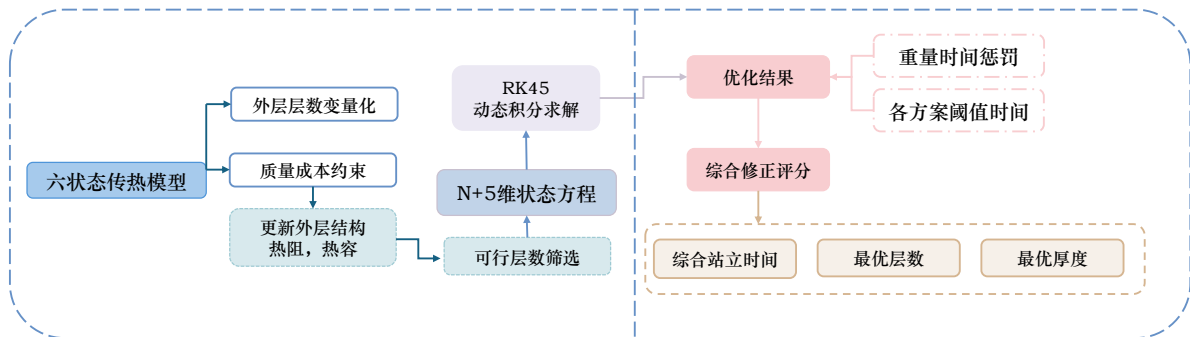


图 12 问题三建模与求解流程图

6.2 多外层涂层传热模型

6.2.1 决策变量与状态向量

设外层涂层总层数为 $N \in \mathbb{Z}_+$ ，原防护服已有一层，则新增层数为 $N - 1$ ，外层总厚度为

$$\delta_{\text{out}}(N) = N\delta_3 = 0.3N \text{ mm}. \quad (31)$$

为同时表示外层总热阻和总热容，对每层涂层分别设置一个平均温度节点。问题三的状态向量扩展为

$$\mathbf{x}_N = \left(T_c, T_s, T_1, T_2, T_{3,1}, \dots, T_{3,N}, \dot{V}_{bl} \right)^T, \quad (32)$$

其维数为 $N + 5$ 。

6.2.2 外层节点的能量平衡

PCM 层中心与第一个外层节点之间的单位面积热阻为

$$r_{2,3_1} = \frac{\delta_2}{2k_2} + \frac{\delta_3}{2k_3},$$

相邻两个外层节点中心之间的热阻为

$$r_{3_j,3_{j+1}} = \frac{\delta_3}{k_3}, \quad j = 1, 2, \dots, N - 1.$$

因此，相邻外层节点之间的热流密度为

$$q_{3_j,3_{j+1}} = \frac{T_{3,j} - T_{3,j+1}}{r_{3_j,3_{j+1}}}.$$

设单层涂层质量为 m_3 ，显热热容为 $C_3 = m_3 c_3$ ，则第 j 个外层节点满足

$$C_3 \frac{dT_{3,j}}{dt} = A_h (q_{\text{in},j} - q_{\text{out},j}). \quad (33)$$

最外节点与环境之间继续采用问题一的自然对流边界：

$$h_{\text{out}} = 2.38 \lambda_{\text{out}} |T_{3,N} - T_{\infty}|^{0.25},$$

$$q_{3,N,\infty} = \frac{T_{3,N} - T_{\infty}}{\delta_3 / (2k_3) + 1 / h_{\text{out}}}.$$

6.3 质量、成本与承重约束

服装材料面积为 $A_m = 1.25A_h = 2.065125 \text{ m}^2$ 。由外层密度 $\rho_3 = 300 \text{ kg/m}^3$ 和单层厚度 $\delta_3 = 0.3 \text{ mm}$ ，得单层质量

$$m_3 = \rho_3 A_m \delta_3 = 0.18586125 \text{ kg}.$$

外层材料单价为 300 元/kg，因此每新增一层的直接材料费为

$$\Delta C_3 = 300 m_3 = 55.758375 \text{ 元}.$$

原防护服的材料成本为 $C_0 = 377.091825$ 元，成本上限为

$$C_{\max} = 1.5 C_0 = 565.6377375 \text{ 元}.$$

层数为 N 时，总质量和总成本分别为

$$m(N) = m_1 + m_2 + N m_3, \quad (34)$$

$$C(N) = C_0 + (N - 1) \Delta C_3. \quad (35)$$

硬约束整套防护服质量，即

$$m(N) \leq 100 \text{ kg}.$$

将式(35)代入成本约束，可得预算可行的整数集合为

$$N \in \{1, 2, 3, 4\}.$$

6.4 重量惩罚与优化模型

设问题一原防护服的最长时间为 $t_0 = t_{15}(1)$ ，增厚带来的纯传热时长增益为 $t_{15}(N) - t_0$ 。题目给出每多承受 0.5 kg 使站立时间减少 10 s，因而时间—重量换算系数为

$$\lambda_m = \frac{10}{0.5} = 20 \text{ s/kg}.$$

时间惩罚以问题一为基准，只计入新增质量

$$\Delta m(N) = m(N) - m(1),$$

因此，问题三的综合站立时间为

$$J(N) = t_0 + [t_{15}(N) - t_0] - \lambda_m \Delta m(N) = t_{15}(N) - 20 [m(N) - m(1)]. \quad (36)$$

最终离散优化模型为

$$\max_N J(N), \quad \text{s.t.} \quad \begin{cases} N \in \{1, 2, 3, 4\}, \\ C(N) \leq 1.5C_0, \\ m(N) \leq 100 \text{ kg}. \end{cases} \quad (37)$$

6.5 模型求解

参照问题一六状态模型的数值求解流程，本文将外层层数枚举嵌入动态传热方程求解，对每个可行方案分别积分，再比较目标函数值。

Step 1: 生成可行方案并设置初始状态。由式(31)、式(34)和式(35)得到候选集合 $N \in \{1, 2, 3, 4\}$ ，逐一计算总厚度、总质量与总成本，筛除不满足预算或承重约束的方案。各方案的核心、皮肤和全部服装节点初温均设为 37°C ，实际皮肤血流初值为 $6.3 \text{ L}/(\text{m}^2\text{h})$ 。

Step 2: 建立并积分变维状态方程。对给定 N ，按式(32)构造 $N + 5$ 维状态向量，并按式(33)更新外层节点数、层间热阻和总显热热容。每次计算方程右端时，先由当前皮肤温度更新目标血流和实际血流变化率，再计算人体两节点、内层、PCM 层及 N 个外层节点的温度导数。基准计算采用自适应 RK45 方法，设置 $\text{rtol} = 10^{-7}$ 、 $\text{atol} = 10^{-9}$ 、最大步长 5 s ，计算上限为 24 h 。

Step 3: 定位温度阈值事件。定义下降方向事件函数

$$g_{15}(t, \mathbf{x}_N) = T_s - 15, \quad g_{10}(t, \mathbf{x}_N) = T_s - 10. \quad (38)$$

求解器利用连续延拓定位式(38)中两个事件的首次过零时刻，得到 t_{15} 和 t_{10} ，不从离散输出点插值反推。为检验数值精度，另以 $\text{rtol} = 10^{-9}$ 、 $\text{atol} = 10^{-11}$ 和最大步长 1 s 对最优候选方案加严复算。

Step 4: 计算评分并选择最优方案。在式(37)给出的可行域内，由式(36)计算 $J(N)$ 。比较四个候选方案的评分，取 $J(N)$ 最大者作为问题三的最优外层厚度。

6.6 结果分析

6.6.1 候选厚度方案

四个可行方案的数值结果如表8所示。

表 8 问题三外层厚度候选方案

N	总厚度/mm	总质量/kg	新增质量/kg	总成本/元	t_{15}/min	惩罚/s	J/min
1	0.3	0.942771	0	377.091825	678.621	0	678.621
2	0.6	1.128632	0.185861	432.850200	696.935	3.717	696.873
3	0.9	1.314493	0.371723	488.608575	715.724	7.434	715.600
4	1.2	1.500355	0.557584	544.366950	735.001	11.152	734.815

由表8可知，外层每增加一层， t_{15} 增加约 18–19 min，而重量惩罚仅增加 3.717 s。保温收益明显大于新增重量代价，因此预算可行域内的最优解为

$$N^* = 4, \quad \delta_{\text{out}}^* = 1.2 \text{ mm}.$$

即在原防护服外层新增 3 层涂层。该方案的纯传热时间为 735.001 min，新增重量惩罚为 11.152 s，所以重量修正后的主指标为 734.815 min。相对问题一基准值 678.621 min，净增益为 56.194 min。

6.7 模型检验与敏感性分析

当 $N = 1$ 时，问题三变维模型退化为问题一六状态模型，两者所得 $t_{15} = 40717.233 \text{ s}$ 完全一致。对四层最优方案加严积分容差并将最大步长由 5 s 减小至 1 s 后， t_{15} 仅改变 $5.11 \times 10^{-4} \text{ s}$ ；基准求解的相对能量守恒误差为 1.05×10^{-7} ，说明层数扩展和事件定位具有良好的数值一致性。

对目标函数的重量惩罚系数扩展分析表明，四层与一层方案发生优劣切换的临界系数约为

$$\lambda_m^* \approx 6067 \text{ s/kg}.$$

只有当系数由题目值 20 s/kg 增大至该量级时，四层方案才会失去优势，因此最优层数对重量惩罚具有很强稳健性。四层方案对未计入的制造附加费用更敏感：若对新增材料成本收取的附加率超过 12.72%，四层方案将超过预算，此时应选择三层方案。

7 问题四模型的建立与求解

7.1 问题分析

问题四要求在不追加材料资金的条件下，仅通过提高中间 PCM 的放热能力，使实验者的室外坚持时间不低于问题三最优方案。由于题目规定 PCM 在各温度下的放热能力同比例提高，因此保持原有相变温区和放热曲线形状不变，引入统一放热倍率对 PCM

相变放热能力进行调整，并以问题三的最长坚持时间作为目标，反演满足要求的最小放热倍率。

7.2 PCM 放热能力倍率反演模型的建立

7.2.1 DSC 潜热峰缩放

设 PCM 基线扣除后的原始放热能力为基准，定义无量纲放热倍率 λ_{pcm} 。将问题一已经建立的核心—皮肤传热、呼吸散热、层间导热、外侧自然对流和皮肤血流动态关系统一写入右端函数，可将问题四的初值问题概括为

$$\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x}; \lambda_{\text{pcm}}). \quad (39)$$

扣除 DSC 基线后，改进后的潜热峰定义为

$$q_{L,\lambda}(T) = \lambda_{\text{pcm}} q_{L,0}(T). \quad (40)$$

7.2.2 单位质量潜热与表观热容

DSC 曲线给出单位质量样品的热流，扫描速率 β 将时间变化转换为温度变化。对改进后的纯相变峰在整个相变温区积分，并完成毫瓦每毫克、分钟和千克之间的单位换算，可得单位质量潜热：

$$L_{\lambda}(\beta) = \frac{60000}{\beta} \int_{T_{\min}}^{T_{\max}} q_{L,\lambda}(T) dT.$$

式中，60000 为单位换算系数， β 的单位为 K/min。该式把离散 DSC 放热数据转化为可进入材料能量平衡的单位质量潜热。将潜热峰缩放关系代入积分，并利用常数倍率可移出积分号的线性性质，可进一步得到

$$L_{\lambda}(\beta) = \lambda_{\text{pcm}} L_0(\beta). \quad (41)$$

在单调降温和单次凝固假设下，潜热释放可等效为相变温区内的附加热容量。将基础比热 c_p 与 DSC 潜热贡献相加，得到改进材料的表观比热：

$$c_{\text{eff},\lambda}(T) = c_p + \frac{60000}{\beta} q_{L,\lambda}(T).$$

式中， c_p 是不参与倍率缩放的基础显热比热；第二项只在相变温区内非零。该表观比热直接决定 PCM 温度节点在吸收或释放相同净热功率时的降温速率。

把 $q_{L,\lambda}(T) = \lambda_{\text{pcm}} q_{L,0}(T)$ 代入上式，可以整理成

$$c_{\text{eff},\lambda}(T) = c_p + \lambda_{\text{pcm}} [c_{\text{eff},0}(T) - c_p]. \quad (42)$$

7.2.3 阈值事件与最小倍率反演模型

题目以人体表面温度低于 15 °C 后工作困难作为比较标准，因此对于给定放热倍率，定义平均皮肤温度首次向下达到该阈值的时间为

$$t_{15}(\lambda_{\text{pcm}}) = \inf \{t \geq 0 : T_s(t; \lambda_{\text{pcm}}) \leq 15^\circ\text{C}\}.$$

问题四没有增加服装质量，所以其比较量直接采用 $t_{15}(\lambda_{\text{pcm}})$ ，不再减去重量惩罚。按照“达到问题三目标且放热倍率尽可能小”的要求，问题四的逆设计模型为

$$\lambda_{\text{pcm}}^* = \min_{\lambda_{\text{pcm}} \geq 1} \{\lambda_{\text{pcm}} : t_{15}(\lambda_{\text{pcm}}) \geq J_3^*\}.$$

7.3 模型求解与结果分析

在正式反演前，本文先对若干低倍率候选值进行事件求解，以检查放热倍率增加时 t_{15} 的变化方向。相邻粗搜索点的 t_{15} 增量均为正，其中最小增量为 30.950405 s。这说明在已检查区间内，提高 PCM 潜热会延后皮肤温度达到 15 °C 的时刻，支持继续向更高倍率扩展根区间并采用单调约束下的标量反演。

Brent 反演得到满足问题三目标的最小 PCM 放热倍率为

$$\lambda_{\text{pcm}}^* = 28.352528,$$

这表示改进材料的纯相变放热能力约为原材料的 28.352528 倍。按照提高百分比的定义，所需新增比例为

$$\eta = (28.352528 - 1) \times 100\% = 2735.253\%.$$

因此，在本文模型及固定其他物性的假设下，中间功能层的放热能力至少需要比原先提高约 2735.25%。

原材料的单位质量潜热为 122.312883 kJ/kg。由潜热与倍率的线性关系，改进后的单位质量潜热为

$$L_{\lambda^*} = 28.352528 \times 122.312883 = 3467.879499 \text{ kJ/kg}.$$

7.3.1 反演曲线与最优倍率分析

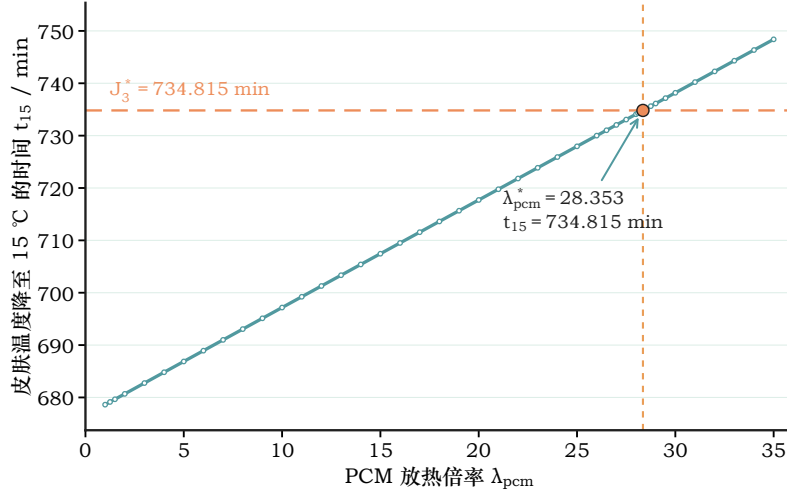


图 13 PCM 放热倍率与皮肤温度阈值时间的反演关系

由图13可见，在已采样范围内， t_{15} 随 λ_{pcm} 增大而持续上升，未出现反向变化；目标水平线与响应曲线在 $\lambda_{\text{pcm}}^* = 28.353$ 处相交，因此该交点给出了满足时间约束的最小可行倍率。原材料在 $\lambda_{\text{pcm}} = 1$ 时的 t_{15} 为 678.621 min，要达到 734.815 min 的目标，需要将纯相变放热能力提高约 2735.25%。这表明在不增加 PCM 质量、厚度且不改变其他物性的条件下，仅依靠提高潜热来补偿约 56.19 min 的坚持时间差，需要很高的

原 PCM 层的总潜热为 55.802491 kJ。由于 PCM 层质量保持不变，最优倍率下整层总潜热增至 1582.141699 kJ。该数值表示 PCM 在一次完整凝固过程中可释放的总能量上限，而不是瞬时放热功率。将最优倍率代回六状态模型，得到

$$t_{15} = 734.814967561 \text{ min}, \quad t_{10} = 1056.377241 \text{ min}.$$

其中 t_{15} 与目标值之差仅为 $7.09 \times 10^{-7} \text{ s}$ ，说明反演解准确落在约束边界上。达到 15°C 皮肤阈值时，核心温度为 26.511699°C，PCM 节点温度为 -19.379422°C，实际皮肤血流量约为 0.610140 L/(m²h)，相变进度已经达到 1。这说明 PCM 潜热在皮肤达到阈值前已经全部释放，后续温度响应仍受人体产热、服装热阻与环境散热共同控制。

7.3.2 参数敏感性与设计稳健性

为判断材料指标对模型参数的依赖程度，本文对内侧换热系数 h_{in} 、基础代谢率 M_0 、外侧自然对流倍率 λ_{out} 和 DSC 扫描速率 β_{DSC} 分别施加上下扰动，并在每个情景下重新反演达到同一目标所需的最小放热倍率。图14给出了最优倍率的归一化敏感度；正值表示参数增大会提高所需倍率，负值表示参数增大可降低所需倍率。

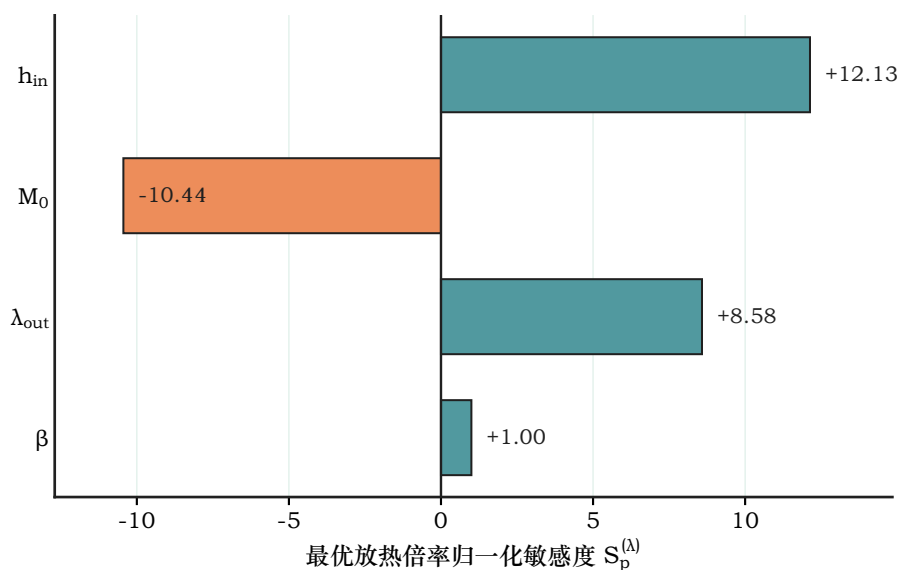


图 14 PCM 最优放热倍率的归一化敏感度

由图14可见， h_{in} 的敏感度为 +12.13，绝对值最大，说明皮肤向服装的换热增强会显著增加 PCM 需要补偿的热量； M_0 的敏感度为 -10.44，代谢产热提高能够减缓人体降温，因而降低所需放热倍率； λ_{out} 的敏感度为 +8.58，表明外界散热增强同样会显著抬高材料性能要求； β_{DSC} 的敏感度为 +1.00，与 DSC 换算潜热随扫描速率反比变化、所需倍率近似同比补偿的关系一致。

8 模型评价

8.1 模型优点

1. 本文将人体热调节与多层防护服传热过程相结合，引入动态皮肤血流和 PCM 相变放热机制，并根据环境风速与人体运动状态修正边界换热和代谢产热条件，较为完整地描述了低温环境下人体-防护服系统的动态传热过程。
2. 针对防护服结构与材料性能优化，本文进一步引入外层厚度、服装质量、材料成本和 PCM 放热能力等因素，在满足承重与资金约束的条件下，分别完成外层结构参数优化和 PCM 最小放热倍率反演，使性能预测模型能够直接服务于防护服设计。

8.2 模型缺点

本文将人体及各层防护服简化为温度均匀的集中参数节点，只描述沿服装厚度方向的平均传热；部分人体热调节与边界换热参数采用经验公式或典型取值，尚未利用实际低温试验进行个体化标定。因此，模型难以反映人体不同部位、局部服装结构及个体生理差异，在复杂低温环境中的预测精度仍有进一步提升空间。

8.3 模型改进与推广

1. 在人体热调节方面，可进一步将人体划分为躯干、四肢等多个热学区域，并引入颤抖产热、个体体脂差异及长时间暴露下的代谢调节机制，以提高核心温度和局部皮肤温度的预测精度。
2. 在防护服传热方面，可将各材料层进一步离散为多个温度节点，并考虑层间接触热阻、服装褶皱、含湿及结霜等因素。重新标定不同工况下的边界换热参数后，该模型可推广至时变风速、交替活动强度及其他极寒作业场景。

参考文献

- [1] 张小雨, 阴建华. 建模分析法在服装热湿舒适性评价中的应用 [J]. 纺织科技进展, 2025, 47(4): 27–32, 40.
- [2] Zheng G, Yue X, Li K, et al. An integrated thermophysiological model for predicting thermal response of medical staff wearing medical protective clothing in summer[J]. International Journal of Thermal Sciences, 2024, 199: 108908.
- [3] 陈雪, 于利静, 张昭华. 个体冷热调节服装的研究进展 [J]. 现代纺织技术, 2024, 32(9): 28–37.
- [4] Cao W, Zhang X, Yao W, et al. Experimental study on the cooling performance of medical protective clothing coupled with phase-change cold storage materials[J]. Applied Thermal Engineering, 2024, 243: 122607.
- [5] 赵荣, 韩子夜, 吴飞翔, 等. 基于十水硫酸钠的个体防护材料的制备及性能 [J]. 材料导报, 2024, 38(3): 22090074–5.
- [6] Gao M, Zhao S, Yang H, et al. An analysis of the influence of DSC parameters on the measurement of the thermal properties of phase-change material[J]. Materials, 2024, 17(23): 5689.
- [7] Zhan R, Zhang B, Liu L, et al. An improved equivalent heat capacity method to simulate and optimize latent thermal energy storage units[J]. International Journal of Heat and Mass Transfer, 2024, 235: 126135.
- [8] 蔡伟熠, 王永臻, 程静, 等. 复合相变调温纺织品研究现状 [J]. 毛纺科技, 2024, 52(12): 130–139.
- [9] Reynolds D R, Gardner D J, Woodward C S, et al. ARKODE: A flexible IVP solver infrastructure for one-step methods[J]. ACM Transactions on Mathematical Software, 2023, 49(2): 1–26.
- [10] 刘叶, 储子晴, 徐广标, 等. 印花织物持续凉感性能评价及影响因素分析 [J]. 东华大学学报 (自然科学版), 2024, 50(4): 9–15.
- [11] 金星, 操凯, 王俊胜, 等. 降温背心的热舒适性研究 [J]. 消防科学与技术, 2024, 43(12): 1737–1744.

- [12] Lakrisenko P, Pathirana D, Weindl D, et al. Benchmarking methods for computing local sensitivities in ordinary differential equation models at dynamic and steady states[J]. PLOS ONE, 2024, 19(10): e0312148.
- [13] 严庆帅, 江潇蒙, 沈华, 等. 个体降温装备研究现状与展望 [J]. 消防科学与技术, 2024, 43(12): 1675–1682.

附录 A 问题一核心传热模型源代码

```
"""Human thermoregulation and clothing heat-transfer equations."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from .config import ModelParameters
from .dsc import PCMMModel

DYNAMIC_SBF = "dynamic_sbf"
QUASISTEADY_SBF = "quasisteady_sbf"
MODEL_VARIANTS = frozenset({DYNAMIC_SBF, QUASISTEADY_SBF})

@dataclass(frozen=True)
class HumanDerivativeResult:
    """Temperature derivatives and mass-redistribution diagnostics."""

    core_temperature_rate_C_s: float
    skin_temperature_rate_C_s: float
    alpha_rate_1_s: float
    mass_core_to_skin_kg_s: float
    enthalpy_mass_transfer_W: float
    mass_redistribution_correction_W: float
    branch: str

def equilibrium_skin_blood_flow(T_skin_C: float | np.ndarray) -> float | np.ndarray:
    """Return equilibrium skin blood flow in L/(m2 h) at skin temperature in degC."""

    temperature = np.asarray(T_skin_C, dtype=float)
    cold_signal = np.maximum(0.0, 33.7 - temperature)
    flow = np.clip(6.3 / (1.0 + 0.5 * cold_signal), 0.5, 90.0)
    return float(flow) if flow.ndim == 0 else flow

def skin_blood_flow(T_skin_C: float | np.ndarray) -> float | np.ndarray:
    """Backward-compatible alias for equilibrium skin blood flow in L/(m2 h)."""

    return equilibrium_skin_blood_flow(T_skin_C)

def d_equilibrium_blood_flow_dTskin(T_skin_C: float) -> float:
    """Return the clip-aware derivative of equilibrium blood flow versus skin temperature."""

    if T_skin_C >= 33.7:
        return 0.0

    denominator = 1.0 + 0.5 * (33.7 - T_skin_C)
    raw_flow = 6.3 / denominator
    if raw_flow <= 0.5 or raw_flow >= 90.0:
        return 0.0
    return 3.15 / denominator**2

def alpha_skin_from_blood_flow(flow_L_m2_h: float | np.ndarray) -> float | np.ndarray:
    """Return the dimensionless Gagge/Pierce skin mass fraction from actual blood flow."""

    flow = np.asarray(flow_L_m2_h, dtype=float)
```

```

alpha = 0.0417737 + 0.7451833 / (flow + 0.585417)
return float(alpha) if alpha.ndim == 0 else alpha

def dalpha_d_blood_flow(flow_L_m2_h: float | np.ndarray) -> float | np.ndarray:
    """Return d(alpha_skin)/d(blood_flow) in (m2 h)/L."""

    flow = np.asarray(flow_L_m2_h, dtype=float)
    derivative = -0.7451833 / (flow + 0.585417) ** 2
    return float(derivative) if derivative.ndim == 0 else derivative

def alpha_skin(T_skin_C: float | np.ndarray) -> float | np.ndarray:
    """Return quasi-steady skin mass fraction as a function of skin temperature."""

    return alpha_skin_from_blood_flow(equilibrium_skin_blood_flow(T_skin_C))

def dalpha_dTskin(T_skin_C: float) -> float:
    """Return clip-aware d(alpha_skin)/d(T_skin) for the quasi-steady model only."""

    flow = float(equilibrium_skin_blood_flow(T_skin_C))
    return float(dalpha_d_blood_flow(flow)) * d_equilibrium_blood_flow_dTskin(T_skin_C)

def respiratory_heat_loss(
    metabolic_rate_W_m2: float, air_temperature_C: float, vapor_pressure_Torr: float
) -> tuple[float, float, float]:
    """Return latent, sensible and total respiratory losses in W/m2."""

    latent = 0.0023 * metabolic_rate_W_m2 * (44.0 - vapor_pressure_Torr)
    sensible = 0.0014 * metabolic_rate_W_m2 * (34.0 - air_temperature_C)
    return latent, sensible, latent + sensible

def _blood_flow_terms(
    state: np.ndarray, params: ModelParameters, model_variant: str
) -> tuple[float, float, float, float]:
    """Return equilibrium flow, actual flow, actual-flow rate and effective tau."""

    if model_variant not in MODEL_VARIANTS:
        raise ValueError(f"Unknown model variant: {model_variant}")
    T_skin = float(state[1])
    equilibrium = float(equilibrium_skin_blood_flow(T_skin))
    if model_variant == DYNAMIC_SBF:
        if len(state) != 6:
            raise ValueError("dynamic_sbf requires a six-state vector")
        actual = float(state[5])
        flow_rate = (equilibrium - actual) / params.blood_flow_time_constant_s
        tau = params.blood_flow_time_constant_s
    else:
        if len(state) != 5:
            raise ValueError("quasisteady_sbf requires a five-state vector")
        actual = equilibrium
        flow_rate = np.nan
        tau = 0.0
    return equilibrium, actual, flow_rate, tau

def heat_fluxes(
    state: np.ndarray, params: ModelParameters, model_variant: str = DYNAMIC_SBF
) -> dict[str, float]:
    """Return heat flux densities in W/m2 using actual blood flow for core-skin transfer."""

```

```

T_core, T_skin, T_layer1, T_pcm, T_layer3 = (float(value) for value in state[:5])
equilibrium_flow, actual_flow, flow_rate, tau = _blood_flow_terms(
    state, params, model_variant
)
G_cs = 5.28 + 1.163 * actual_flow
q_cs = G_cs * (T_core - T_skin)

r_s1 = 1.0 / params.h_in_W_m2K + params.layer1.thickness_m / (
    2.0 * params.layer1.conductivity_W_mK
)
r_12 = params.layer1.thickness_m / (2.0 * params.layer1.conductivity_W_mK) + (
    params.pcm_layer.thickness_m / (2.0 * params.pcm_layer.conductivity_W_mK)
)
r_23 = params.pcm_layer.thickness_m / (2.0 * params.pcm_layer.conductivity_W_mK) + (
    params.layer3.thickness_m / (2.0 * params.layer3.conductivity_W_mK)
)
difference_to_air = T_layer3 - params.air_temperature_C
h_out = params.lambda_out * 2.38 * max(abs(difference_to_air), 1.0e-12) ** 0.25
r_3inf = params.layer3.thickness_m / (2.0 * params.layer3.conductivity_W_mK) + 1.0 / h_out

latent, sensible, total = respiratory_heat_loss(
    params.metabolic_rate_W_m2, params.air_temperature_C, params.vapor_pressure_Torr
)
return {
    "skin_blood_flow_eq_L_m2_h": equilibrium_flow,
    "skin_blood_flow_actual_L_m2_h": actual_flow,
    "skin_blood_flow_L_m2_h": actual_flow,
    "d_skin_blood_flow_dt_L_m2_h_s": flow_rate,
    "tau_blood_flow_s": tau,
    "G_cs_W_m2K": G_cs,
    "q_cs_W_m2": q_cs,
    "q_res_lat_W_m2": latent,
    "q_res_sens_W_m2": sensible,
    "q_res_total_W_m2": total,
    "q_s1_W_m2": (T_skin - T_layer1) / r_s1,
    "q_12_W_m2": (T_layer1 - T_pcm) / r_12,
    "q_23_W_m2": (T_pcm - T_layer3) / r_23,
    "q_3inf_W_m2": difference_to_air / r_3inf,
    "h_out_W_m2K": h_out,
}

def _mass_transfer_result(
    T_core_C: float,
    T_skin_C: float,
    F_core_W: float,
    F_skin_W: float,
    alpha: float,
    alpha_rate_1_s: float,
    params: ModelParameters,
) -> HumanDerivativeResult:
    """Apply upstream-enthalpy mass-transfer branches for a known alpha rate."""

    mb_cb = params.body_mass_kg * params.body_heat_capacity_J_kgK
    C_skin = alpha * mb_cb
    C_core = (1.0 - alpha) * mb_cb
    correction = mb_cb * (T_core_C - T_skin_C) * alpha_rate_1_s
    if alpha_rate_1_s >= 0.0:
        core_rate = F_core_W / C_core
        skin_rate = (F_skin_W + correction) / C_skin
        branch = "core_to_skin" if alpha_rate_1_s > 0.0 else "zero_derivative"
    else:
        skin_rate = F_skin_W / C_skin
        core_rate = (F_core_W + correction) / C_core

```

```

        branch = "skin_to_core"

    mass_rate = params.body_mass_kg * alpha_rate_1_s
    upstream_temperature = T_core_C if mass_rate >= 0.0 else T_skin_C
    enthalpy_transfer = (
        params.body_heat_capacity_J_kgK
        * (upstream_temperature - params.reference_temperature_C)
        * mass_rate
    )
)
return HumanDerivativeResult(
    core_temperature_rate_C_s=core_rate,
    skin_temperature_rate_C_s=skin_rate,
    alpha_rate_1_s=alpha_rate_1_s,
    mass_core_to_skin_kg_s=mass_rate,
    enthalpy_mass_transfer_W=enthalpy_transfer,
    mass_redistribution_correction_W=correction,
    branch=branch,
)

def human_temperature_derivatives_dynamic_sbf(
    T_core_C: float,
    T_skin_C: float,
    actual_flow_L_m2_h: float,
    flow_rate_L_m2_h_s: float,
    F_core_W: float,
    F_skin_W: float,
    params: ModelParameters,
) -> HumanDerivativeResult:
    """Return explicit human temperature rates for the six-state dynamic-SBF model."""

    alpha = float(alpha_skin_from_blood_flow(actual_flow_L_m2_h))
    alpha_rate = float(dalpha_d_blood_flow(actual_flow_L_m2_h)) * flow_rate_L_m2_h_s
    return _mass_transfer_result(
        T_core_C, T_skin_C, F_core_W, F_skin_W, alpha, alpha_rate, params
    )

def human_temperature_derivatives_quasisteady_sbf(
    T_core_C: float,
    T_skin_C: float,
    F_core_W: float,
    F_skin_W: float,
    params: ModelParameters,
) -> HumanDerivativeResult:
    """Solve the implicit conservative five-state quasi-steady-SBF branches."""

    alpha = float(alpha_skin(T_skin_C))
    derivative = dalpha_dTskin(T_skin_C)
    mb_cb = params.body_mass_kg * params.body_heat_capacity_J_kgK
    C_skin = alpha * mb_cb
    C_core = (1.0 - alpha) * mb_cb
    if derivative == 0.0:
        core_rate = F_core_W / C_core
        skin_rate = F_skin_W / C_skin
        alpha_rate = 0.0
    else:
        coupling = mb_cb * (T_core_C - T_skin_C) * derivative
        candidate_skin_rate = F_skin_W / (C_skin - coupling)
        candidate_alpha_rate = derivative * candidate_skin_rate
        if candidate_alpha_rate >= 0.0:
            core_rate = F_core_W / C_core
            skin_rate = candidate_skin_rate
            alpha_rate = candidate_alpha_rate

```

```

        else:
            skin_rate = F_skin_W / C_skin
            alpha_rate = derivative * skin_rate
            core_rate = (F_core_W + mb_cb * (T_core_C - T_skin_C) * alpha_rate) / C_core
        result = _mass_transfer_result(
            T_core_C, T_skin_C, F_core_W, F_skin_W, alpha, alpha_rate, params
        )
    )
    return HumanDerivativeResult(
        core_temperature_rate_C_s=core_rate,
        skin_temperature_rate_C_s=skin_rate,
        alpha_rate_i_s=result.alpha_rate_i_s,
        mass_core_to_skin_kg_s=result.mass_core_to_skin_kg_s,
        enthalpy_mass_transfer_W=result.enthalpy_mass_transfer_W,
        mass_redistribution_correction_W=result.mass_redistribution_correction_W,
        branch=result.branch,
    )

def human_temperature_derivatives(
    T_core_C: float,
    T_skin_C: float,
    F_core_W: float,
    F_skin_W: float,
    params: ModelParameters,
) -> HumanDerivativeResult:
    """Backward-compatible wrapper for the five-state quasi-steady human model."""

    return human_temperature_derivatives_quasisteady_sbf(
        T_core_C, T_skin_C, F_core_W, F_skin_W, params
    )

def state_diagnostics(
    state: np.ndarray,
    params: ModelParameters,
    pcm: PCMMModel,
    model_variant: str = DYNAMIC_SBF,
) -> dict[str, float | str]:
    """Return instantaneous diagnostics for an explicitly selected blood-flow model."""

    T_core, T_skin, _, T_pcm, _ = (float(value) for value in state[:5])
    flux = heat_fluxes(state, params, model_variant)
    actual_flow = float(flux["skin_blood_flow_actual_L_m2_h"])
    alpha = float(alpha_skin_from_blood_flow(actual_flow))
    mb_cb = params.body_mass_kg * params.body_heat_capacity_J_kgK
    F_core = params.heat_transfer_area_m2 * (
        params.metabolic_rate_W_m2 - flux["q_res_total_W_m2"] - flux["q_cs_W_m2"]
    )
    F_skin = params.heat_transfer_area_m2 * (
        flux["q_cs_W_m2"] - flux["q_si_W_m2"]
    )
    if model_variant == DYNAMIC_SBF:
        human = human_temperature_derivatives_dynamic_sbf(
            T_core,
            T_skin,
            actual_flow,
            float(flux["d_skin_blood_flow_dt_L_m2_h_s"]),
            F_core,
            F_skin,
            params,
        )
    else:
        human = human_temperature_derivatives_quasisteady_sbf(
            T_core, T_skin, F_core, F_skin, params

```

```

    )
    flux["d_skin_blood_flow_dt_L_m2_h_s"] = (
        d_equilibrium_blood_flow_dTskin(T_skin)
        * human.skin_temperature_rate_C_s
    )

    phase_fraction = float(pcm.phase_fraction_released(T_pcm))
    latent_heat = pcm.latent_heat_J_kg(params.dsc_scan_rate_K_min)
    c_eff = float(pcm.effective_heat_capacity_J_kgK(T_pcm, params.dsc_scan_rate_K_min))
    area = params.heat_transfer_area_m2
    layer1_rate = area * (flux["q_s1_W_m2"] - flux["q_12_W_m2"]) / params.layer1_capacity_J_K
    pcm_rate = area * (flux["q_12_W_m2"] - flux["q_23_W_m2"]) / (
        params.pcm_mass_kg * c_eff
    )
    layer3_rate = area * (flux["q_23_W_m2"] - flux["q_3inf_W_m2"]) / params.layer3_capacity_J_K
    return {
        **flux,
        "alpha_skin": alpha,
        "alpha_dot_1_s": human.alpha_rate_1_s,
        "C_core_J_K": (1.0 - alpha) * mb_cb,
        "C_skin_J_K": alpha * mb_cb,
        "c_eff_pcm_J_kgK": c_eff,
        "pcm_phase_fraction": phase_fraction,
        "pcm_latent_released_J": params.pcm_mass_kg * latent_heat * phase_fraction,
        "dm_core_to_skin_kg_s": human.mass_core_to_skin_kg_s,
        "enthalpy_mass_transfer_W": human.enthalpy_mass_transfer_W,
        "mass_redistribution_correction_W": human.mass_redistribution_correction_W,
        "human_mass_branch": human.branch,
        "dT_core_dt_C_s": human.core_temperature_rate_C_s,
        "dT_skin_dt_C_s": human.skin_temperature_rate_C_s,
        "dT_layer1_dt_C_s": layer1_rate,
        "dT_pcm_dt_C_s": pcm_rate,
        "dT_layer3_dt_C_s": layer3_rate,
    }

def rhs_quasisteady_sbf(
    _time_s: float, state: np.ndarray, params: ModelParameters, pcm: PCModel
) -> np.ndarray:
    """Return five temperature derivatives for the quasi-steady-SBF model."""

    diagnostics = state_diagnostics(state, params, pcm, QUASISTEADY_SBF)
    return np.array(
        [
            diagnostics["dT_core_dt_C_s"],
            diagnostics["dT_skin_dt_C_s"],
            diagnostics["dT_layer1_dt_C_s"],
            diagnostics["dT_pcm_dt_C_s"],
            diagnostics["dT_layer3_dt_C_s"],
        ],
        dtype=float,
    )

def rhs_dynamic_sbf(
    _time_s: float, state: np.ndarray, params: ModelParameters, pcm: PCModel
) -> np.ndarray:
    """Return five temperature derivatives plus actual skin-blood-flow derivative."""

    diagnostics = state_diagnostics(state, params, pcm, DYNAMIC_SBF)
    return np.array(
        [
            diagnostics["dT_core_dt_C_s"],
            diagnostics["dT_skin_dt_C_s"],

```

```

        diagnostics["dT_layer1_dt_C_s"],
        diagnostics["dT_pcm_dt_C_s"],
        diagnostics["dT_layer3_dt_C_s"],
        diagnostics["d_skin_blood_flow_dt_L_m2_h_s"],
    ],
    dtype=float,
)

def rhs(
    time_s: float,
    state: np.ndarray,
    params: ModelParameters,
    pcm: PCModel,
    model_variant: str = DYNAMIC_SBF,
) -> np.ndarray:
    """Dispatch to an explicitly named five- or six-state model variant."""

    if model_variant == DYNAMIC_SBF:
        return rhs_dynamic_sbf(time_s, state, params, pcm)
    if model_variant == QUASISTEADY_SBF:
        return rhs_quasisteady_sbf(time_s, state, params, pcm)
    raise ValueError(f"Unknown model variant: {model_variant}")

```

附录 B 问题二有风轻微运动模型源代码

```

"""Problem 2 six-state heat-transfer equations with wind forcing."""

from __future__ import annotations

import numpy as np

from src.问题一.dsc import PCModel
from src.问题一.model import (
    alpha_skin_from_blood_flow,
    dalphi_d_blood_flow,
    equilibrium_skin_blood_flow,
    human_temperature_derivatives_dynamic_sbf,
    respiratory_heat_loss,
)

from .config import ModelParameters

CORE_SKIN_BASE_CONDUCTANCE_W_M2K = 5.28
CORE_SKIN_FLOW_COEFFICIENT_W_H_L_K = 1.163
NATURAL_CONVECTION_COEFFICIENT = 2.38
FORCED_CONVECTION_COEFFICIENT = 12.1

def heat_fluxes(state: np.ndarray, params: ModelParameters) -> dict[str, float]:
    """Return Problem 2 heat fluxes and outside convection coefficients."""

    if len(state) != 6:
        raise ValueError("Problem 2 requires the six-state vector [Tc, Ts, T1, T2, T3, Vbl]")
    T_core, T_skin, T_layer1, T_pcm, T_layer3, actual_flow = (
        float(value) for value in state
    )
    equilibrium_flow = float(equilibrium_skin_blood_flow(T_skin))
    flow_rate = (

```

```

        equilibrium_flow - actual_flow
    ) / params.blood_flow_time_constant_s
    conductance = (
        CORE_SKIN_BASE_CONDUCTANCE_W_M2K
        + CORE_SKIN_FLOW_COEFFICIENT_W_H_L_K * actual_flow
    )

    r_s1 = 1.0 / params.h_in_W_m2K + params.layer1.thickness_m / (
        2.0 * params.layer1.conductivity_W_mK
    )
    r_12 = params.layer1.thickness_m / (2.0 * params.layer1.conductivity_W_mK) + (
        params.pcm_layer.thickness_m / (2.0 * params.pcm_layer.conductivity_W_mK)
    )
    r_23 = params.pcm_layer.thickness_m / (2.0 * params.pcm_layer.conductivity_W_mK) + (
        params.layer3.thickness_m / (2.0 * params.layer3.conductivity_W_mK)
    )
    difference_to_air = T_layer3 - params.air_temperature_C
    h_nat = NATURAL_CONVECTION_COEFFICIENT * max(abs(difference_to_air), 1.0e-12) ** 0.25
    h_forced = FORCED_CONVECTION_COEFFICIENT * np.sqrt(params.wind_speed_m_s)
    h_out = max(h_nat, h_forced)
    r_3inf = params.layer3.thickness_m / (2.0 * params.layer3.conductivity_W_mK) + 1.0 / h_out
    respiratory_latent, respiratory_sensible, respiratory_total = respiratory_heat_loss(
        params.metabolic_rate_W_m2,
        params.air_temperature_C,
        params.vapor_pressure_Torr,
    )
    return {
        "skin_blood_flow_eq_L_m2_h": equilibrium_flow,
        "skin_blood_flow_actual_L_m2_h": actual_flow,
        "d_skin_blood_flow_dt_L_m2_h_s": flow_rate,
        "tau_blood_flow_s": params.blood_flow_time_constant_s,
        "G_cs_W_m2K": conductance,
        "q_cs_W_m2": conductance * (T_core - T_skin),
        "q_res_lat_W_m2": respiratory_latent,
        "q_res_sens_W_m2": respiratory_sensible,
        "q_res_total_W_m2": respiratory_total,
        "q_s1_W_m2": (T_skin - T_layer1) / r_s1,
        "q_12_W_m2": (T_layer1 - T_pcm) / r_12,
        "q_23_W_m2": (T_pcm - T_layer3) / r_23,
        "q_3inf_W_m2": difference_to_air / r_3inf,
        "h_nat_W_m2K": h_nat,
        "h_forced_W_m2K": h_forced,
        "h_out_W_m2K": h_out,
    }

def state_diagnostics(
    state: np.ndarray, params: ModelParameters, pcm: PCMModel
) -> dict[str, float | str]:
    """Return all state-dependent rates, capacities, fluxes and PCM diagnostics."""

    T_core, T_skin, _, T_pcm, _, actual_flow = (float(value) for value in state)
    flux = heat_fluxes(state, params)
    alpha = float(alpha_skin_from_blood_flow(actual_flow))
    flow_rate = float(flux["d_skin_blood_flow_dt_L_m2_h_s"])
    alpha_rate = float(dalpha_d_blood_flow(actual_flow)) * flow_rate
    area = params.heat_transfer_area_m2
    F_core = area * (
        params.metabolic_rate_W_m2
        - float(flux["q_res_total_W_m2"])
        - float(flux["q_cs_W_m2"])
    )
    F_skin = area * (float(flux["q_cs_W_m2"]) - float(flux["q_s1_W_m2"]))
    human = human_temperature_derivatives_dynamic_sbf(

```

```

    T_core,
    T_skin,
    actual_flow,
    flow_rate,
    F_core,
    F_skin,
    params,
)
c_eff = float(
    pcm.effective_heat_capacity_J_kgK(T_pcm, params.dsc_scan_rate_K_min)
)
phase_fraction = float(pcm.phase_fraction_released(T_pcm))
latent_heat = pcm.latent_heat_J_kg(params.dsc_scan_rate_K_min)
mb_cb = params.body_mass_kg * params.body_heat_capacity_J_kgK
return {
    **flux,
    "alpha_skin": alpha,
    "d_alpha_skin_dt_1_s": alpha_rate,
    "C_core_J_K": (1.0 - alpha) * mb_cb,
    "C_skin_J_K": alpha * mb_cb,
    "c_eff_pcm_J_kgK": c_eff,
    "pcm_phase_fraction": phase_fraction,
    "pcm_latent_released_J": params.pcm_mass_kg * latent_heat * phase_fraction,
    "dm_core_to_skin_kg_s": human.mass_core_to_skin_kg_s,
    "enthalpy_mass_transfer_W": human.enthalpy_mass_transfer_W,
    "mass_redistribution_correction_W": human.mass_redistribution_correction_W,
    "human_mass_branch": human.branch,
    "dT_core_dt_C_s": human.core_temperature_rate_C_s,
    "dT_skin_dt_C_s": human.skin_temperature_rate_C_s,
    "dT_layer1_dt_C_s": area
    * (float(flux["q_s1_W_m2"]) - float(flux["q_12_W_m2"]))
    / params.layer1_capacity_J_K,
    "dT_pcm_dt_C_s": area
    * (float(flux["q_12_W_m2"]) - float(flux["q_23_W_m2"]))
    / (params.pcm_mass_kg * c_eff),
    "dT_layer3_dt_C_s": area
    * (float(flux["q_23_W_m2"]) - float(flux["q_3inf_W_m2"]))
    / params.layer3_capacity_J_K,
}

def rhs(_time_s: float, state: np.ndarray, params: ModelParameters, pcm: PCMModel) -> np.ndarray:
    """Return the derivatives of the formal Problem 2 six-state ODE."""

    diagnostics = state_diagnostics(state, params, pcm)
    return np.array(
        [
            diagnostics["dT_core_dt_C_s"],
            diagnostics["dT_skin_dt_C_s"],
            diagnostics["dT_layer1_dt_C_s"],
            diagnostics["dT_pcm_dt_C_s"],
            diagnostics["dT_layer3_dt_C_s"],
            diagnostics["d_skin_blood_flow_dt_L_m2_h_s"],
        ],
        dtype=float,
    )

```

附录 C 问题三多外层传热模型源代码

```

"""Variable-dimension thermal model for multiple outer insulation coatings."""

```

```

from __future__ import annotations

import numpy as np

from src.问题一.dsc import PCMMModel
from src.问题一.model import (
    alpha_skin_from_blood_flow,
    dalpha_d_blood_flow,
    equilibrium_skin_blood_flow,
    human_temperature_derivatives_dynamic_sbf,
    respiratory_heat_loss,
)

from .config import Problem3Parameters

def state_dimension(outer_layer_count: int) -> int:
    """Return core, skin, inner, PCM, outer nodes and actual blood-flow dimension."""

    if outer_layer_count < 1:
        raise ValueError("At least one outer coating is required")
    return outer_layer_count + 5

def initial_state(config: Problem3Parameters, outer_layer_count: int) -> np.ndarray:
    """Return the initial state with one temperature node per outer coating."""

    thermal = config.thermal
    outer = [thermal.initial_layer3_temperature_C] * outer_layer_count
    return np.array(
        [
            thermal.initial_core_temperature_C,
            thermal.initial_skin_temperature_C,
            thermal.initial_layer1_temperature_C,
            thermal.initial_pcm_temperature_C,
            *outer,
            thermal.initial_skin_blood_flow_L_m2_h,
        ],
        dtype=float,
    )

def state_diagnostics(
    state: np.ndarray,
    config: Problem3Parameters,
    pcm: PCMMModel,
    outer_layer_count: int,
) -> dict[str, float | str | np.ndarray]:
    """Return heat fluxes, human regulation and all state derivatives."""

    if len(state) != state_dimension(outer_layer_count):
        raise ValueError("State dimension does not match outer coating count")

    params = config.thermal
    T_core = float(state[0])
    T_skin = float(state[1])
    T_inner = float(state[2])
    T_pcm = float(state[3])
    T_outer = np.asarray(state[4 : 4 + outer_layer_count], dtype=float)
    actual_flow = float(state[-1])

    equilibrium_flow = float(equilibrium_skin_blood_flow(T_skin))
    flow_rate = (

```

```

    equilibrium_flow - actual_flow
) / params.blood_flow_time_constant_s
alpha = float(alpha_skin_from_blood_flow(actual_flow))
alpha_rate = float(dalpha_d_blood_flow(actual_flow)) * flow_rate

G_cs = 5.28 + 1.163 * actual_flow
q_cs = G_cs * (T_core - T_skin)
r_s1 = 1.0 / params.h_in_W_m2K + params.layer1.thickness_m / (
    2.0 * params.layer1.conductivity_W_mK
)
r_i2 = params.layer1.thickness_m / (
    2.0 * params.layer1.conductivity_W_mK
) + params.pcm_layer.thickness_m / (
    2.0 * params.pcm_layer.conductivity_W_mK
)
r_pcm_outer = params.pcm_layer.thickness_m / (
    2.0 * params.pcm_layer.conductivity_W_mK
) + params.layer3.thickness_m / (
    2.0 * params.layer3.conductivity_W_mK
)
r_outer_interface = params.layer3.thickness_m / params.layer3.conductivity_W_mK

q_s1 = (T_skin - T_inner) / r_s1
q_i2 = (T_inner - T_pcm) / r_i2
q_pcm_outer = (T_pcm - T_outer[0]) / r_pcm_outer
if outer_layer_count > 1:
    q_outer_interfaces = (T_outer[:-1] - T_outer[1:]) / r_outer_interface
else:
    q_outer_interfaces = np.empty(0, dtype=float)

difference_to_air = float(T_outer[-1] - params.air_temperature_C)
h_out = params.lambda_out * 2.38 * max(abs(difference_to_air), 1.0e-12) ** 0.25
r_outer_air = params.layer3.thickness_m / (
    2.0 * params.layer3.conductivity_W_mK
) + 1.0 / h_out
q_outer_air = difference_to_air / r_outer_air

q_res_lat, q_res_sens, q_res_total = respiratory_heat_loss(
    params.metabolic_rate_W_m2,
    params.air_temperature_C,
    params.vapor_pressure_Torr,
)
area = params.heat_transfer_area_m2
F_core = area * (params.metabolic_rate_W_m2 - q_res_total - q_cs)
F_skin = area * (q_cs - q_s1)
human = human_temperature_derivatives_dynamic_sbf(
    T_core,
    T_skin,
    actual_flow,
    flow_rate,
    F_core,
    F_skin,
    params,
)

c_eff = float(
    pcm.effective_heat_capacity_J_kgK(T_pcm, params.dsc_scan_rate_K_min)
)
inner_rate = area * (q_s1 - q_i2) / params.layer1_capacity_J_K
pcm_rate = area * (q_i2 - q_pcm_outer) / (params.pcm_mass_kg * c_eff)
outer_rates = np.empty(outer_layer_count, dtype=float)
for index in range(outer_layer_count):
    incoming = q_pcm_outer if index == 0 else float(q_outer_interfaces[index - 1])
    outgoing = q_outer_air if index == outer_layer_count - 1 else float(q_outer_interfaces[index])

```

```

        outer_rates[index] = (
            area * (incoming - outgoing) / config.outer_coating_capacity_J_K
        )

mb_cb = params.body_mass_kg * params.body_heat_capacity_J_kgK
phase_fraction = float(pcm.phase_fraction_released(T_pcm))
latent_heat = pcm.latent_heat_J_kg(params.dsc_scan_rate_K_min)
derivatives = np.array(
    [
        human.core_temperature_rate_C_s,
        human.skin_temperature_rate_C_s,
        inner_rate,
        pcm_rate,
        *outer_rates,
        flow_rate,
    ],
    dtype=float,
)
)
return {
    "skin_blood_flow_eq_L_m2_h": equilibrium_flow,
    "skin_blood_flow_actual_L_m2_h": actual_flow,
    "d_skin_blood_flow_dt_L_m2_h_s": flow_rate,
    "tau_blood_flow_s": params.blood_flow_time_constant_s,
    "alpha_skin": alpha,
    "alpha_dot_1_s": alpha_rate,
    "C_core_J_K": (1.0 - alpha) * mb_cb,
    "C_skin_J_K": alpha * mb_cb,
    "G_cs_W_m2K": G_cs,
    "q_cs_W_m2": q_cs,
    "q_res_lat_W_m2": q_res_lat,
    "q_res_sens_W_m2": q_res_sens,
    "q_res_total_W_m2": q_res_total,
    "q_s1_W_m2": q_s1,
    "q_12_W_m2": q_12,
    "q_pcm_outer_W_m2": q_pcm_outer,
    "q_outer_interfaces_W_m2": q_outer_interfaces,
    "q_outer_air_W_m2": q_outer_air,
    "h_out_W_m2K": h_out,
    "c_eff_pcm_J_kgK": c_eff,
    "pcm_phase_fraction": phase_fraction,
    "pcm_latent_released_J": params.pcm_mass_kg * latent_heat * phase_fraction,
    "dm_core_to_skin_kg_s": human.mass_core_to_skin_kg_s,
    "enthalpy_mass_transfer_W": human.enthalpy_mass_transfer_W,
    "mass_redistribution_correction_W": human.mass_redistribution_correction_W,
    "human_mass_branch": human.branch,
    "state_derivatives": derivatives,
}

def rhs(
    _time_s: float,
    state: np.ndarray,
    config: Problem3Parameters,
    pcm: PCModel,
    outer_layer_count: int,
) -> np.ndarray:
    """Return the variable-dimension Problem 3 state derivative."""

    return np.asarray(
        state_diagnostics(state, config, pcm, outer_layer_count)["state_derivatives"],
        dtype=float,
    )

```

附录 D 问题四放热倍率反演源代码

```
"""Continuous-event solves and scalar Brent inversion for Problem 4."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq

from ..config import ModelParameters
from ..dsc import PCMMModel
from ..model import DYNAMIC_SBF, rhs
from .config import SolverSettings
from .pcm import ScaledPCMMModel

@dataclass(frozen=True)
class EventResult:
    lambda_pcm: float
    t15_s: float
    t10_s: float
    event_15_reached: bool
    event_10_reached: bool

@dataclass
class InverseResult:
    lambda_opt: float
    target_t15_s: float
    evaluations: dict[float, float]
    bracket: tuple[float, float]
    root_method: str = "brentq"

def _initial_state(params: ModelParameters) -> np.ndarray:
    return np.array(
        [
            params.initial_core_temperature_C,
            params.initial_skin_temperature_C,
            params.initial_layer1_temperature_C,
            params.initial_pcm_temperature_C,
            params.initial_layer3_temperature_C,
            params.initial_skin_blood_flow_L_m2_h,
        ],
        dtype=float,
    )

def solve_events(
    params: ModelParameters,
    base_pcm: PCMMModel,
    lambda_pcm: float,
    settings: SolverSettings,
    *,
    stop_at_15: bool = False,
) -> EventResult:
    """Solve with continuous downward 15/10 degC event detection."""

    pcm = ScaledPCMMModel(base_pcm, lambda_pcm)
```

```

def event15(_time_s: float, state: np.ndarray) -> float:
    return float(state[1] - 15.0)

def event10(_time_s: float, state: np.ndarray) -> float:
    return float(state[1] - 10.0)

event15.direction = -1.0
event15.terminal = bool(stop_at_15)
event10.direction = -1.0
event10.terminal = True
events = (event15,) if stop_at_15 else (event15, event10)
solution = solve_ivp(
    fun=lambda time, state: rhs(time, state, params, pcm, DYNAMIC_SBF),
    t_span=(0.0, settings.horizon_s),
    y0=initial_state(params),
    method=settings.method,
    rtol=settings.rtol,
    atol=settings.atol,
    max_step=settings.max_step_s,
    events=events,
)
if not solution.success:
    raise RuntimeError(f"Problem 4 event solve failed: {solution.message}")
t15 = float(solution.t_events[0][0]) if len(solution.t_events[0]) else np.nan
if stop_at_15:
    t10 = np.nan
else:
    t10 = float(solution.t_events[1][0]) if len(solution.t_events[1]) else np.nan
return EventResult(
    lambda_pcm=lambda_pcm,
    t15_s=t15,
    t10_s=t10,
    event_15_reached=bool(np.isfinite(t15)),
    event_10_reached=bool(np.isfinite(t10)),
)

def invert_lambda(
    params: ModelParameters,
    base_pcm: PCMMModel,
    target_t15_s: float,
    settings: SolverSettings,
    *,
    initial_upper: float = 1.5,
    known_evaluations: dict[float, float] | None = None,
) -> InverseResult:
    """Find the constrained minimum multiplier that reaches the target."""

    cache = dict(known_evaluations or {})

    def evaluate(lambda_pcm: float) -> float:
        key = float(lambda_pcm)
        if key not in cache:
            event = solve_events(
                params, base_pcm, key, settings, stop_at_15=True
            )
            if not event.event_15_reached:
                raise RuntimeError(f"15 degC event not reached at lambda={key}")
            cache[key] = event.t15_s
        return cache[key]

    t_at_one = evaluate(1.0)
    if t_at_one >= target_t15_s:
        return InverseResult(1.0, target_t15_s, cache, (1.0, 1.0))

```

```

upper = max(float(initial_upper), 1.0 + 1.0e-6)
while evaluate(upper) < target_t15_s:
    upper *= 1.5
    if upper > settings.maximum_lambda:
        raise RuntimeError(
            f"Could not bracket target below lambda={settings.maximum_lambda}"
        )
lower_candidates = [
    value
    for value, t15_s in cache.items()
    if value >= 1.0 and value < upper and t15_s < target_t15_s
]
lower = max(lower_candidates, default=1.0)
root = brentq(
    lambda value: evaluate(float(value)) - target_t15_s,
    lower,
    upper,
    xtol=settings.root_xtol,
    rtol=max(4.0 * np.finfo(float).eps, settings.root_xtol),
)
evaluate(float(root))
return InverseResult(float(root), target_t15_s, cache, (lower, upper))

```